



End-to-End Autonomous Navigation using Homography-Based Overhead Localization

**John Christian Niño T. Abuel
Rey Iann Tigley
Kervin Lemuel Paalisbo**

Supervised by
Asst Prof. Renato V. Crisostomo, MSCS

Department of Computer Science
College of Computer Studies
MSU - Iligan Institute of Technology

Jan, 2026

*A undergraduate thesis proposal submitted in partial fulfilment of the
requirements for the degree of B.S. in Computer Science.*



Copyright ©2026 MSU - Iligan Institute of Technology

www.msuiit.edu.ph

Abstract

Traditional autonomous navigation systems usually depend on expensive multi-sensor fusion or computationally intensive deep learning architectures that process raw, ego-centric video feeds. These high resource requirements create constraints to deploying scalable, low-cost robotics platforms in educational or industrial settings. To address these computational and hardware limitations, a lightweight end-to-end navigation framework is proposed that effectively bridges geometric computer vision with imitation learning. The architecture utilizes a single RGB camera and ground-plane fiducial markers to dynamically generate a homography matrix, transforming perspective-distorted frames into rectified, top-down Birds-Eye-View (BEV) images. These geometrically simplified inputs are subsequently fed into a streamlined Convolutional Neural Network (CNN) trained via behavioral cloning to map path features directly to continuous steering angles. The system is rigorously evaluated across straight, complex turn, and recovery scenarios to assess stability and generalization without PID feedback. Performance is quantified using Root Mean Square Error (RMSE) for tracking accuracy, heading error for orientation, and an oscillation index to measure steering smoothness. The approach demonstrates that integrating classical homography techniques with deep learning significantly reduces computational overhead, offering a feasible alternative to heavy SLAM-based methods for structured indoor environments.

Keywords: Autonomous Navigation, Convolutional Neural Networks, Homography, Behavioral Cloning, Mobile Robots, Computer Vision

Contents

1	Introduction	1
1.1	Background of the Study	1
1.2	Statement of the Problem	3
1.3	Research Objective	3
1.3.1	General Objective	3
1.3.2	Specific Objectives	3
1.4	Scope and Limitations of the Research	4
1.5	Significance of the Research	5
2	Literature Review	7
2.1	Search Strategy	7
2.2	Inclusion and Exclusion Criteria	7
2.3	Results	8
2.4	Autonomous Navigation and Sensor Technologies	9
2.4.1	Monocular and Multicamera RGB Systems	10
2.4.2	RGB-D Cameras	11
2.4.3	LiDAR and Camera-LiDAR Hybrids	12
2.4.4	Event-Based Cameras	12
2.4.5	Infrared and Thermal Imaging Sensors	12
2.4.6	Comparative Summary	13
2.5	Computer Vision for Autonomous Robot Systems	14
2.5.1	LiDar and RTK-GPS and its Limitations	14
2.5.2	Fiducial Marker Methods	15
2.5.3	Marker-Based Localization, Mapping, and ArUco Systems	16

2.5.4	Modern Detection and Real-Time Systems	19
2.6	Deep Learning Approaches for Robot Control Strategies	19
2.6.1	Deep Learning Scopes	20
2.6.2	Deep Learning Techniques	21
2.7	Theoretical Framework	22
2.7.1	Projective Geometry and Planar Homography	22
2.7.2	Neural Networks and Convolutional Neural Networks (CNNs) .	23
2.7.3	Imitation Learning and Behavioral Cloning	24
3	Methodology	26
3.1	Hardware Setup	26
3.1.1	Robot Platform	26
3.1.2	Environment Setup	27
3.2	Software Framework	27
3.2.1	Core Libraries	28
3.2.2	Camera and System Calibration	28
3.2.3	Perception and Localization Module	29
3.2.4	ArUco Marker Detection	30
3.2.5	Planar Homography Estimation	31
3.3	System Integration and Control Flow	31
3.3.1	Module Integration	32
3.3.2	Real-Time Execution and Middleware	36
3.4	Data Acquisition and Processing	36
3.4.1	Dataset Collection	37
3.4.2	State Representation	39
3.5	Convolutional Neural Network Model and Training	41
3.5.1	Model Architecture	41
3.5.2	Training Configuration	43
3.5.3	Deployment and Inference	44
3.6	Evaluation and Testing	45
3.6.1	Physical Testing Environment and Setup	45
3.6.2	Evaluation Scenarios	45
3.6.3	Ground-Truth Tracking System	46
3.6.4	Performance Metrics	47
4	Results & Discussion	50

4.1	Chronological Development and Experimental Baselines	50
4.1.1	Iteration 1: Baseline A (Allocentric Full AOI Model)	51
4.1.2	Iteration 2: Baseline B (Feature-Extracted Allocentric Model) . . .	51
4.2	The Egocentric Dynamic Crop Model	56
4.2.1	Offline Training Convergence	56
4.2.2	Real-World Navigation and Endurance	58
4.3	Comparative Tracking Performance	58
4.4	In-Depth Performance of the Model	60
4.4.1	Statistical Analysis Methodology	61
4.4.2	Per-Lap Tracking Consistency	62
4.4.3	Scenario 1: 45-Degree Bézier Curves	63
4.4.4	Scenario 2: 90-Degree Bézier Corners (Square Path)	63
4.4.5	Scenario 3: 180-Degree Sustained U-Turns	64
4.5	Control Decision Discretization and Confusion Matrix	66
Appendix A Calendar of Activities		69
Appendix B Personal Vitae		70

Introduction

Rather than relying on manually tuned classical controllers, this study implements an end-to-end control strategy where a single neural network is responsible for the entire navigation pipeline, from perception to actuation. By preprocessing the environment using ArUco-based localization, the 'intelligence' is shifted from complex feature extraction to direct motion planning. This eliminates the latency and complexity associated with tuning PID gains, resulting in a potentially better architecture where the robot learns to interpret geometric spatial relationships and execute smooth motor responses simultaneously.

1.1 | Background of the Study

Autonomous navigation is the ability of robots and autonomous vehicles to generate a path and move along it without human intervention, utilizing data captured from onboard sensors and sometimes remote navigation aids [1]. These systems consistently rely on simultaneous localization and mapping (SLAM) for autonomous navigation. Additionally, these systems also utilize LiDAR, depth cameras, or multi-sensor fusion [2]. While generally effective, these robotic systems demand high computational costs, expensive robotics components, and complex software integration requirements, which makes them less accessible for developing low-cost robotics platforms [3]. To address these limitations, previous studies have already explored computer-vision navigation using fiducial markers, which offer a simple, lightweight, and reliable framework for localization, particularly in structured environments [4].

According to [5], a fiducial marker refers to a specialized code-type marker used in

computer vision, which is set in an environment or scene to help imaging systems find reference points. The robustness of fiducial markers such as ArUco and AprilTag in providing low-cost localization has already been explored in a 2025 systematic review of the mapping of mobile robot navigation (?). The study of ? concluded that marker size, spacing, and the placement of a fiducial marker may affect robot navigation, well-designed markers show potential in achieving high success rates with minimal computational overhead.

Several recent studies probed the ability of ArUco markers to provide indoor localization for autonomous robot navigation. Research on planar indoor localization using ArUco ? and multi-marker setups illustrates that planar marker fields can deliver accurate 2-D pose estimates which are appropriate for autonomous robot navigation (?), especially when combined with simple filters or odometry. To further maximize localization, multi-camera setups are also explored, as seen in the study of ?. Studies on fiducial-marker networks for indoor navigation (?) explicitly frame markers as an alternative to the expensive SLAM for building service and inspection robots, highlighting low instrumentation and computation as key advantages.

For robot navigation, recent research has shown interest in using end-to-end frameworks that map visual input directly to steering (?) or obstacle avoidance actions, usually with deep reinforcement learning or attention-based architecture (??). Although such methods can achieve strong obstacle avoidance performance in simulation or structured test environments, multiple studies emphasize issues with stability, overfitting, and susceptibility to noise, leading to poor generalization outside training conditions.

Current research in End-to-End Visual Navigation (e.g., NVIDIA's PILOTNet) focuses predominantly on unstructured environments using raw, ego-centric camera feeds. Because these systems must implicitly learn to interpret depth, perspective, and 3D geometry from raw pixels, they require deep, computationally expensive Convolutional Neural Networks (CNNs). This requires high-performance onboard hardware (GPUs), which significantly increases system cost and complexity.

There is a distinct lack of research that leverages geometric constraints to simplify this learning process. Most existing studies either use fiducial markers (like ArUco) solely for coordinate-based localization or rely on heavy deep learning models for raw visual perception. Few have explored bridging these domains by using markers to pre-process the visual feed.

This study addresses this gap by proposing a lightweight architecture where ArUco-based homography is used to transform raw video frames into rectified, top-down (Birds-Eye-View) images. By feeding these geometrically simplified images, rather than raw perspective views, into the neural network, the complexity of the learning task is greatly reduced. This allows a standard CNN to map visual inputs directly to steering angles with high accuracy, enabling robust end-to-end autonomous navigation on significantly constrained hardware.

1.2 | Statement of the Problem

This study aims to address the lack of an accessible, low-cost autonomous navigation framework for small indoor mobile robots that relies solely on camera input and operates in a structured 2-D environment, since existing SLAM-based solutions depend on expensive sensors, high computational resources, and complex software pipelines, while current computer-vision and self-driving approaches either overlook planar ArUco-based localization or fail to integrate it with end-to-end strategies in a way that is suitable for resource-constrained platforms.

1.3 | Research Objective

The following are the general objective and specific objectives of the study.

1.3.1 | General Objective

To develop an autonomous vehicle system utilizing an end-to-end Convolutional Neural Network (CNN) architecture that maps raw visual input directly to steering and throttle actuation signals, eliminating the need for intermediate feedback control algorithms (PID).

1.3.2 | Specific Objectives

1. To implement a robust Inverse Perspective Mapping (IPM) pipeline using ArUco markers for dynamic homography calibration, capable of transforming angled, ego-centric RGB camera frames into rectified, top-down, bird's-eye view (BEV) representations of the local path.

2. To construct a specialized behavioral cloning dataset consisting of synchronized pairs of preprocessed BEV path images and human-demonstrated steering angles, collected across various track configurations.
3. To design and train a regression-based Convolutional Neural Network (CNN) architecture that maps high-dimensional BEV visual inputs directly to continuous steering commands (and optionally linear velocity), utilizing Mean Squared Error (MSE) loss for optimization.
4. To deploy the trained model onto the vehicle's onboard computer (e.g., Raspberry Pi or Jetson) such that the inference output is sent directly to the motor drivers without corrective feedback or error-calculation layers.
5. To test the vehicle's ability to navigate a course autonomously, measuring success based on lane-keeping accuracy and smoothness of motion, and comparing these metrics against the original human training data.

1.4 | Scope and Limitations of the Research

This study focuses on the development of a low-cost, end-to-end autonomous navigation system using a single onboard RGB camera. The system utilizes Inverse Perspective Mapping (IPM) facilitated by ArUco markers to process visual inputs.

The hardware implementation is restricted to computationally constrained platforms (e.g., Raspberry Pi or NVIDIA Jetson Nano) to validate the lightweight nature of the proposed architecture. The robot operates exclusively in structured environments where ArUco markers can be reliably detected, and no 3-D mapping, LiDAR, or depth-based perception is included. Unlike traditional SLAM systems that integrate probabilistic mapping and multi-sensor fusion ??, this study does not attempt full environment reconstruction or dynamic obstacle mapping. Instead, localization is constrained to planar marker fields consistent with research demonstrating accurate 2-D pose estimation using fiducials (??).

As for the environmental scope of this study, the navigation environment is limited to structured, planar indoor surfaces with consistent artificial lighting. The robot operates within an area of interest, which is a predefined 2-D plane where ArUco markers are visible and explicitly placed to define track boundaries or reference points.

The system is not designed for outdoor environments, uneven terrain, or varying illumination conditions.

Neural-network steering prediction is trained only on data collected within the experimental environment. Thus, the learned model is not expected to generalize to unstructured or markerless environments, consistent with known limitations of end-to-end visual navigation models regarding generalization and noise sensitivity (??). The evaluation metrics, path accuracy, time to goal, and success rate, are limited to controlled indoor trials and do not include long-term autonomy, multi-robot coordination, or obstacle-rich navigation.

1.5 | Significance of the Research

This study contributes to the growing demand for accessible and low cost autonomous driving robot navigation frameworks by presenting a system that removes the need for expensive sensors, advanced SLAM algorithms, and high-end computing hardware. While previous works have highlighted that fiducial markers such as Aruco provide efficient localization (??), relatively few studies have used these markers into a complete low cost 2-D navigation system suited for educational or research purposes.

This research contributes to the field of robotic control by exploring the potential of End-to-End Imitation Learning in constrained environments. While previous works have validated ArUco markers for localization (?) and CNNs for visual navigation separately, this study bridges these domains. It investigates whether a homography-based pre-processing step can simplify visual inputs enough to allow a lightweight neural network to provide navigation controls accurately without the computational overhead of deeper, complex architectures.

The study is locally and globally significant because it provides:

1. This study provides a framework for implementing these advanced Deep Learning concepts using only a standard webcam and a microcontroller, making the technology accessible for educational and low-resource industrial applications.
2. A simple experimental platform for teaching concepts related to localization, control systems, and machine learning.

3. A replicable and scalable framework for researchers exploring low-cost autonomous navigation in structured indoor spaces.

Overall, this study aims to address the gaps in the literature by producing a fully integrated, low-cost navigation framework that combines fiducial-marker localization and End-to-End robot control, offering a practical solution for accessible autonomous robot navigation systems.

Literature Review

This chapter reviews the existing literature on autonomous mobile recognition systems, with a focus on sensor technologies, marker based localization, and robot control strategies. It analyzes the advantages and limitations of current methodologies to establish the theoretical framework for the proposed low-cost navigation system. Additionally, the Preferred Reporting Items for Systematic Reviews and Meta-Analysis (PRISMA) guidelines was done to ensure a systematic and transparent process of searching, screening, and inclusion of literature.

2.1 | Search Strategy

To begin the literature review process and identify relevant studies, the researchers conducted a literature search across Google Scholar, IEEE Xplore, and arXiv. The search was done specifically with the query strings shown in Table 2.1 below. Query strings were varied for each database to ensure that the strings were syntactically correct with each database's unique search syntax.

2.2 | Inclusion and Exclusion Criteria

The search process yielded a total of 2050 papers but narrowed them down to 15 papers by selecting them based on the inclusion and exclusion criteria shown in Table 2.2.

The papers from all databases were pooled together, and the screening process started. Titles and abstracts were screened to identify studies relevant to the research topic.

DATABASE	Query String
Google Scholar	("ground robot" OR "mobile robot" OR UGV OR "autonomous robot") "vision-based localization" OR "fiducial marker" OR "ArUco" OR "AprilTag" "path planning" OR "trajectory tracking" "neural network" OR "pure pursuit" OR PID
IEEE Xplore	("ground robot" OR "mobile robot" OR "autonomous robot") AND ("vision-based localization" OR "computer vision" OR "fiducial marker" OR "ArUco" OR "AprilTag") AND ("pose estimation" OR homography) AND ("path planning" OR "coverage path planning" OR "trajectory tracking") AND ("pure pursuit" OR PID OR "neural network" OR "learning-based control")
ScienceDirect	("ArUco" OR "AprilTag" OR "fiducial marker") AND ("pose estimation" OR SLAM) AND ("mobile robot" OR robotics) AND (camera OR vision)

Table 2.1: Databases and query strings used in the literature search.

Inclusion	Exclusion
■ Papers published between 2020 and 2025. ■ No paywall. ■ Uses fiducial markers or overhead-based camera localization. ■ Includes mobile robots. ■ Uses Neural Network or Hybrid (Classical + NN) Frameworks or purely classical framework.	■ Studies older than 6 years. ■ Papers with paywall. ■ Studies without using fiducial markers or any camera vision. ■ Studies focusing only on SLAM without fiducial markers, overhead vision, or robot control relevance. ■ Non-ground robots unless used for overhead vision.

Table 2.2: Inclusion and exclusion criteria for literature selection.

2.3 | Results

The systematic review and screening process identified a total of 58 relevant papers after gathering results from all of the databases. All studies were then assessed for eligibility based on the inclusion and exclusion criteria.

During the abstract and title screening, studies that did not use ground mobile robots, fiducial markers, overhead camera, or were otherwise irrelevant to autonomous navigation were excluded. At the full-text assessment stage, a total of 43 studies were excluded due to the following reasons:

- No use of fiducial markers or overhead camera
- UAV-only studies where the primary subject was aerial
- Surveys, reviews, datasets, or patents with no experimental results
- Studies without relevant navigation or control frameworks

After the full-text assessment, a final number of 15 papers were included in the qualitative review. These studies used ground mobile robots, fiducial markers, overhead camera-based localization, and also added a neural, hybrid, or purely classical control frameworks for navigation. A prisma flow diagram summarizing this process is presented in Figure 2.1 below. Lastly, in addition to the studies included in our systematic review (PRISMA), other relevant literature, especially the ones provided by the adviser of this research, was also considered to provide context and background for this study.

2.1

2.4 | Autonomous Navigation and Sensor Technologies

Autonomous navigation fundamentally relies on a mobile robot's ability to perceive its environment and determine its position relative to the environment. While autonomous navigation is a well established field, the hardware requirements for reliable performance remain a significant barrier for academic and low cost applications. In our review of literature on autonomous navigation, we found out that modern autonomous navigation relies on a heterogeneous sensor suite. Some of the most widely adopted technologies range from the classic RGB-D cameras, to innovations like event based sensors (??). Recent multi-sensor approaches such as the work of ?, who integrated LiDAR, YOLO-based AprilTag detection, and depth cues for autonomous docking, demonstrate how heterogeneous sensor fusion significantly

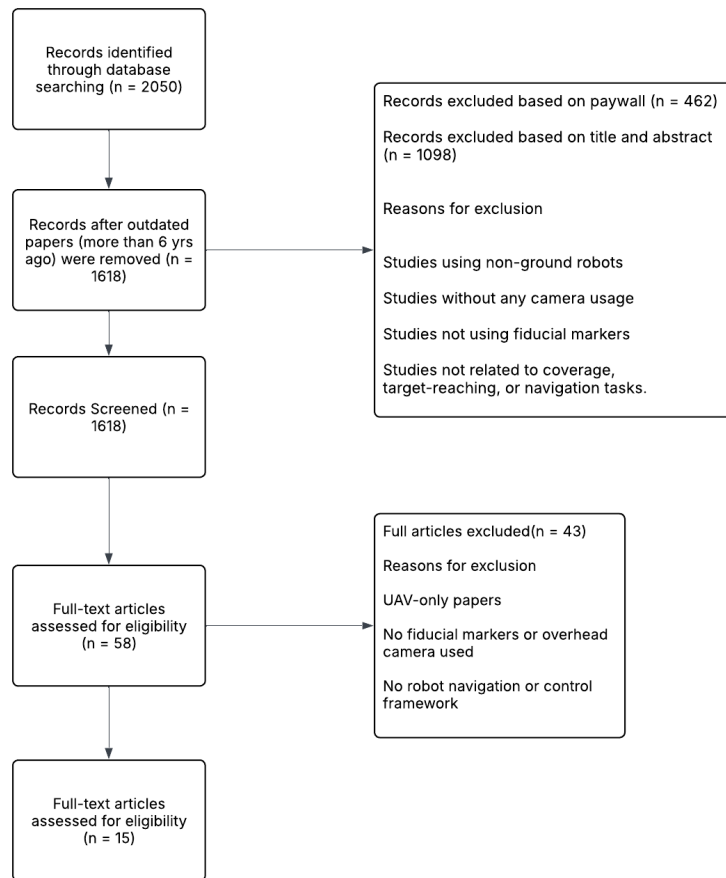


Figure 2.1: PRISMA 2020 Flowchart

improves docking precision and reliability in cluttered indoor environments. In this subchapter, we evaluate the advantages and disadvantages of each sensor technology.

2.4.1 | Monocular and Multicamera RGB Systems

Monocular RGB cameras remain one of the most widely used sensing modalities in autonomous navigation due to their low cost, size, and ease of integration. Recent work demonstrates that deep learning greatly improves the capability of monocular systems, enabling better perception even without clear depth sensors. Research has shown that convolutional and transformer-based networks can infer scene geometry, obstacle layout, and traversability directly from RGB images (?). Additional monocular systems, such as the neural-network-driven visual servoing framework by ?, show

how wide-FOV fisheye lenses combined with CNN-based controllers can enhance robustness even when depth cues are limited. Similarly, ? demonstrated that pose-based visual servoing enhanced with lightweight deep-learning binarization improves tracking accuracy in resource-limited mobile robot platforms. These methods reduce hardware complexity while maintaining acceptable localization performance for indoor and structured environments. Despite these advantages, monocular cameras suffer from an inherent depth ambiguity that can limit reliability in dynamic or cluttered spaces. Studies on monocular systems trained with deep reinforcement learning highlight that navigation performance largely depends on the variety of training environments (?). To help solve the depth limitation, recent research combines multiple deep learning modules and depth prediction to create better feature representations for navigation, as shown by ?. These hybrid pipelines increase robustness but require more computational resources than classical monocular methods.

2.4.2 | RGB-D Cameras

While Monocular and multi-camera RGB systems provide useful visual data for navigation, their lack of direct depth information restricts their robustness, especially in tasks requiring reliable 3D localization or obstacle avoidance. RGB-D cameras solve this problem by combining regular RGB images with depth information, producing a richer representation that simplifies tasks such as pose estimation, mapping, and collision detection. According to ?, RGB-D sensors are good alternatives to traditional range sensors like LiDAR because they are often cheap, compact, and require low-power, yet still provide both color and depth data that support 3D scene reconstruction. RGB-D sensing also appears in navigation pipelines such as the omnidirectional manufacturing-oriented robot proposed by ?, where combined RGB-D perception enables reliable autonomous movement in industrial indoor environments.

Many state-of-the-art approaches combine RGB-D data with odometry / SLAM pipelines to exploit this richer input. For instance, ? demonstrated a practical indoor navigation system using fused inputs from a 2D LiDAR and an RGB-D camera. This hybrid setup allowed a real mobile robot to navigate without a precomputed map. On the other hand, recent work such as ? shows how modern deep-learning techniques can further improve performance: by refining noisy or incomplete depth maps (common with low-cost Kinect-like sensors), the study achieved more reliable indoor visual odometry under budget-friendly constraints.

2.4.3 | LiDAR and Camera-LiDAR Hybrids

Recent studies highlight that combining LiDAR with cameras significantly improves autonomous navigation systems. According to ?, combining data from LiDAR and cameras lead to more accurate and robust mapping and localization, than just relying on either modality alone. There are studies that support this view, for example, ?, designed a framework that combined the use of LiDAR and wide-angle camera images and it resulted in significantly improved free-space detection performance. Hybrid LiDAR-vision architectures further align with the findings of ?, who showed that fusing CNN-based visual relocalization with progressive LiDAR scan-matching significantly enhances robustness under viewpoint changes and degraded visual conditions. However, these papers also highlight the limitation of the LiDAR-camera fusion which requires precise calibration and consistent environmental conditions that make real-time deployment more computationally demanding.

2.4.4 | Event-Based Cameras

Event-based cameras output streams of “events” rather than fixed video frames, because of this, they offer compelling advantages for robotics and SLAM. According to ?, event cameras deliver high temporal resolution, low latency, reduced power consumption and data bandwidth compared to traditional frame-based cameras. Additionally, the hardware architecture of this type of camera can efficiently handle the asynchronous event streams which enable low-power SLAM and perception on embedded robotic systems.

2.4.5 | Infrared and Thermal Imaging Sensors

Infrared and thermal imaging sensors give strong advantages for autonomous robots especially in visually degraded and low-light environments. ? showed that thermal imaging provides reliable localization for robots even in smoke, dust, or foggy conditions where performance of other cameras like RGB and LiDAR deteriorates. Similarly, recent work on Thermal SLAM by ?. demonstrates that temperature-based features can be used for object detection and tracking which improves localization regardless of lighting.

2.4.6 | Comparative Summary

Sensing Modality	Key Advantages	Major Limitations
Monocular and Multicamera RGB	■ Low cost, compact size, and ease of integration. ■ Simplifies mapping and collision detection.	■ Prone to noise or incomplete depth maps. ■ Often limited to indoor ranges.
LiDAR & Camera Hybrids	■ Significantly improved free-space detection. ■ More accurate and robust mapping than the single standalone of each part.	■ Requires precise calibration. ■ Computationally demanding for real-time deployment. ■ Requires environmental consistency.
Event-Based Cameras	■ High temporal resolution and low latency. ■ Reduced power consumption and data bandwidth.	■ Requires specific hardware architectures. ■ Output is non-traditional (events) which requires specific processing.
Infrared & Thermal Sensors	■ Reliability in visually degraded conditions. ■ Effective even in low-light, low visibility environments.	■ Primarily focused on visually degraded conditions. ■ Less feature rich for standard navigation.

Table 2.3: Sensing modalities, advantages, and limitations.

As shown in Table 2.3, while LiDAR and Hybrid systems offer high accuracy, they suffer from high computational demands and calibration complexity. Conversely,

Monocular RGB systems (Section 2.2.1) offer a balance of low cost and hardware simplicity. By leveraging Deep Learning—specifically the Neural Network architecture inherent in the DonkeyCar framework—this study aims to mitigate the ‘depth ambiguity’ limitation of monocular cameras by supplementing local vision with a global overhead supervisor (ArUco), thereby maintaining the low-resource benefits of RGB systems while improving navigation reliability.

2.5 | Computer Vision for Autonomous Robot Systems

The computer vision systems in robotics systems serve as the eyes of the robot – providing vision that helps it recognize and understand its surroundings. Studies such as [?] also show that camera-only visual localization pipelines can remain effective even in complex, GPS-limited outdoor urban environments. Computer vision as defined by [?], is the technique where robotic systems use cameras and sensors to gather visual information. Then the information gathered is then processed by algorithms, which enable robots to perform complex tasks like assembly, quality control, and navigation.

2.5.1 | LiDar and RTK-GPS and its Limitations

LiDar and RTK-GPS (Real-Time Kinematic GPS), alone or in combination, are widely used to produce accurate mapping for vehicles ([?]). However, there are several limitations that push for the implementation of alternative approaches for autonomous systems.

2.5.1.1 | Cost, Hardware, and Computational Requirements

LiDar systems, as described by [?], are often expensive and sizable for some applications. [?] supports this idea in their study, concluding that LiDar is the most promising method for producing accurate, real-time localization among other approaches but suffers from the limitations of the system’s computational ability, requiring more powerful CPU/GPU. This limitation poses a problem for smaller systems such as mobile robots where money, weight, and power consumption directly impact the navigation operations.

2.5.1.2 | GPS Signal Degradation and Denial

RTK-GPS systems may suffer from extreme performance degradation or complete signal loss in a constrained environment such as urban canyons, underground or multi-level parking structures, tunnels, and indoor spaces (???). Odometry errors accumulate fast without external correction, leading to inaccuracy in position estimates. ? resorted to alternative methods for a localization of self-driving racecars because GPS can be unreliable on racing tracks and in harsh environments.

The limitations for LiDar and GPS collectively motivate the exploration of modern sensing modalities that can provide reliable localization in challenging scenarios where traditional systems struggle (??). This necessitates a need for developing alternative low-cost computer-vision approaches based on artificial fiducial markers that can provide reliable positioning without expensive sensors.

2.5.2 | Fiducial Marker Methods

Fiducial markers are designed visual patterns which are machine or camera-detectable strategically placed within an environment to provide reliable localization mapping or pose estimation cues for autonomous systems. These visual markers serve as the landmarks that may offer multiple advantages over resource-intensive, natural or feature-based localization techniques. A broader mapping of marker-based navigation techniques is summarized in the systematic review by ?, which highlights the lightweight computational requirements, reliability, and adaptability of fiducial marker systems across diverse robotic domains.

In comparative studies of standard, custom, reflective, and infrared markers, fiducial markers generally offer high detection accuracy, robustness, and low computational cost, but their performance can still degrade under poor lighting conditions ?. Standard physical markers will include but not limited to, AR Tag (?), AprilTag (?), and Aruco markers (?).

2.5.2.1 | Operational Advantages

When compared to localization methods like GPS RTK, the pragmatic advantages behind physical fiducial markers come from their ability to provide discrete, absolute position fixes that can correct accumulated drift in dead-reckoning systems (??)

Markers also facilitate the creation of persistent maps with known landmark

positions. Unlike natural feature-based SLAM, which must continuously track and update potentially ambiguous environmental features, marker-based systems can rely on the stable, unambiguous identity and position of installed markers. This stability simplifies map maintenance and enables more predictable localization performance (??).

2.5.2.2 | Applications in Autonomous Systems

Fiducial markers were leveraged to localization for vehicle parking in a GPS-denied setting like an underground parking lot for precise autonomous driving (?). Moreover, indoor autonomous drones use ceiling or wall mounted markers for positioning during warehouse inventory and structural examination tasks (?). Marker systems have also been applied in assistive indoor navigation, such as the ARUCO-based pan-tilt camera system demonstrated by ? for improving mobility support for blind and visually impaired users. Overhead or airborne localization frameworks similarly leverage markers, as seen in ?, who demonstrated a UAV-based visual navigation method using squared planar fiducials for orientation and path-following.

These applications share a common theme: they operate in environments where GPS is unavailable or unreliable, infrastructure modification is feasible, and cost-effective localization solutions are essential. Fiducial markers address these themes by providing a pragmatic middle ground between expensive LiDAR systems and drift-prone dead-reckoning approaches. The following section (2.5.3) details how these fiducial marker principles are realized in marker-based localization and mapping architectures, with a focus on ArUco-based systems.

2.5.3 | Marker-Based Localization, Mapping, and ArUco Systems

Building on the general properties of fiducial markers outlined in Section 2.2.2, this section examines marker-based localization and mapping pipelines, emphasizing ArUco-based implementations in autonomous vehicles and robots.

2.5.3.1 | Detection and Pose Estimation

Planar fiducial marker systems usually follow a structured pipeline that begins with image acquisition and proceeds through marker detection, corner localization, and pose estimation. In common implementations such as ArUco, candidate markers are extracted by thresholding and contour detection, followed by decoding of the internal

binary pattern to recover marker identity and reject false positives (?). Once the four marker corners are localized in image coordinates and associated with a known marker size, the perspective-n-point (PnP) algorithm is used together with the camera calibration parameters to obtain the 6-degrees-of-freedom (6-DoF) transformation between the marker frame and the camera frame (??). This geometric formulation avoids any need for training data, and it provides an interpretable pose estimate that can be directly integrated into downstream localization and mapping pipelines.

2.5.3.2 | Robot and Camera Pose Tracking with ArUco

Real-time implementations such as the UAV-mounted camera system in ? confirm that marker-based pose tracking remains feasible even under dynamic motion, non-static camera poses, and rapid changes in scene geometry. Additionally, lightweight educational platforms, such as those in the Duckietown initiative by ?, highlight how careful camera calibration and wheel-encoder integration support accurate pose estimation even on low-cost robotic systems. When fiducial markers are deployed in the environment, each ArUco detection yields a relative pose measurement between the camera and the corresponding marker. Assuming a calibrated camera and known marker dimensions, these measurements can be expressed in metric units and transformed into a robot-centric or world-centric frame, enabling continuous tracking of the robot's position and orientation as it moves through the environment (??). In practice, the system accumulates a time series of marker-based pose updates, which are combined with kinematic information such as wheel odometry to produce a smooth estimate of the robot trajectory in GPS-denied settings.

2.5.3.3 | ArUco-based Planar Task-Space Localization

Aside from providing global or relative pose measurements, planar fiducial configurations can be used to define local task spaces through homography estimation (?). Numerous experimental systems have demonstrated the practicality of ArUco markers for autonomous vehicle and robot localization.

By placing multiple ArUco markers at the corners of a region of interest, ? explored an optimized system that can compute a projective transformation between image coordinates and a planar world coordinate frame aligned with the physical surface. In addition, ? follows this idea for indoor drone tracking through markers on walls that define a room coordinate system (inside-out tracking). This demonstrates the efficacy of fiducial markers to provide planar task-space localization. ? applied this system for

docking stations to achieve robust positioning with low-cost cameras and embedded processors.

2.5.3.4 | Sensor Fusion Architectures

In conventional autonomous systems, marker-based pose estimates are typically fused with proprioceptive sensors to compensate for intermittent visibility and measurement noise (?). Extended Kalman filters, error-state Kalman filters, and particle filters are widely used to combine discrete, absolute pose fixes from fiducial markers with continuous estimates derived from wheel encoders and inertial measurement units (??). Marker–odometry fusion has also been demonstrated effectively by ?, who showed that integrating AprilTag observations with particle filtering reduces drift and improves localization accuracy in indoor autonomous robots. In systems like these, the prediction step propagates the robot state forward using odometry or motion models, while the update step incorporates new marker observations when available, bounding drift and improving robustness when there is sensor noise and environmental disturbance.

2.5.3.5 | Marker-aided SLAM and Mapping

In recent meta, mapping for autonomous robots are commonly carried out through LiDAR vision, which is costly and complex when it comes to maintenance ?. Traditional Marker-aided SLAM formulations incorporate fiducial markers as explicit landmarks in the map, rather than relying solely on natural point or line features ?. In graph-based SLAM, markers are represented as nodes with fixed or estimated poses, and edges encode relative constraints between robot poses and marker observations; optimization of this graph improves both robot trajectories and marker maps ?. This approach simplifies data association, because each marker carries a unique identifier, and supports persistent, globally consistent maps that can be reused across sessions, which is particularly advantageous in structured environments such as parking facilities and industrial halls.

Recent implementations reinforce the practical value of this paradigm. For example, ? demonstrated a real-time camera-based SLAM pipeline capable of mapping unknown indoor environments without the use of LiDAR, showing that lightweight vision systems can be competitive when supported by robust tracking and optimization. Marker-enhanced localization pipelines also appear in aerial–ground hybrid systems such as ?, where a tethered UAV and ground robot perform collaborative localization using deep neural networks supplemented by intermittent fiducial marker detections.

These studies highlight how marker-aided SLAM can reduce cost, simplify data association through unique IDs, and enable persistent, reusable maps which are particularly beneficial in structured indoor environments.

2.5.3.6 | Gap Analysis for Marker-Based Detection and Localization

While the literature establishes that marker-based sensor fusion and SLAM architectures (discussed in Sections 2.5.3.4 and 2.5.3.5) provide robust state estimation, they inherently introduce significant computational complexity and latency due to the iterative nature of Kalman filtering and graph optimization. Moreover, these traditional pipelines rely heavily on explicit state estimation. This requires the system to calculate the exact coordinates at every time step before making control decisions. This dependency introduces architectural complexity and computational overhead. Section 2.6 reviews recent advancements in deep learning for robot control, providing further justification for the shift away from traditional pose estimation toward end-to-end architectures.

2.5.4 | Modern Detection and Real-Time Systems

Autonomous robots require vision-based localization pipelines that operate under strict real-time and resource constraints, often on embedded platforms such as single-board computers or low-power system-on-chip devices. The detection and pose estimation of fiducial markers must run at sufficient frame rates to support closed-loop control, while sharing computational resources with other perception and control modules. This has motivated a line of work focused on optimizing fiducial detection algorithms, memory usage, and numerical routines so that marker-based localization remains feasible on platforms with limited CPU, GPU, and power budgets ?.

2.6 | Deep Learning Approaches for Robot Control Strategies

The integration of neural networks in autonomous systems have shifted the industry standard from hand-coded features, to data driven methodologies. In the context of autonomous mobile robot navigation, deep learning techniques have been increasingly used to process complex sensory data and generate navigational decisions. Upon reviewing literature, a survey from ? shows three deep learning paradigms for

robot navigation; End-to-End Learning (E2E), Learning Subsystems, and Learning Components. Additionally, specific neural network architectures and training methodologies, referred to here as deep learning techniques, are employed across these paradigms to handle varied input modalities and control objectives. Recent marker- and vision-based navigation studies reinforce this trend. For instance, ? demonstrated that CNN-based relocalization can significantly improve pose recovery within a hybrid LiDAR-vision localization framework, showing how learned perception compensates for degraded geometric features. Deep neural controllers have also been applied to monocular systems, such as the visual servoing approach by ?, which uses fisheye camera imagery to control a wheeled mobile robot. Similarly, ? leveraged lightweight deep-learning binarization to enhance pose-based visual servoing for resource-constrained mobile robots. Hybrid aerial-ground systems such as ? also illustrate how DNNs can infer relative positions between robots when fiducial marker signals are intermittent or partially occluded. Together, these systems demonstrate that deep learning serves not only as a replacement for classical control but also as a complementary module for perception, relocalization, and decision-making within modern autonomous navigation architectures.

2.6.1 | Deep Learning Scopes

The application of deep learning in autonomous mobile robot navigation is categorized by how much of the classical navigation stack (perception, localization, planning, and control) is replaced by neural networks. If the whole system is replaced with a neural network (E2E), if only subsystems of the navigation system are replaced (Learning Subsystems), or if only components (Learning Components).

End-to-End (E2E) Learning seeks to replace the classical navigation stack with a single deep neural network that maps raw inputs directly to control actions. This approach centralizes the process by integrating it into a single neural network process. ? demonstrated this by developing a deep learning model that processes RGB camera data to output steering in a single step. Similarly, ? applied E2E learning to robotic following behaviors, proving that a single network could learn complex tracking tasks without explicit object recognition modules.

Deep Learning Components, unlike E2E, involve replacing a single processing step within a module with a deep learning model. The most common application is in the perception module, where Convolutional Neural Networks (CNNs) replace manual

feature extractors to perform object detection or semantic segmentation. For example, ? introduced YOLO, a real-time object detection system that serves as a perception component, feeding bounding box coordinates to a classical costmap or planner. In this scope, the neural network does not make navigational decisions; it strictly provides structured information to a classical algorithm.

Deep Learning Subsystems replace an entire subprocess in the multi-step process of the autonomous robot navigation stack, while retaining the classical methods for the other parts. One such example is ? DeepVO architecture proposal. The deep learning architecture replaces the classical visual odometry system with a Recurrent Convolutional Neural Network (RCNN). This subsystem takes raw video as input and outputs the robot's pose, directly replacing the localization module while leaving the planning and control modules untouched. A similar study from ? proposed "PRM-RL," which replaces the local path planner subsystem with a Reinforcement Learning agent to handle dynamic obstacle avoidance, while still relying on a classical Probabilistic Roadmap (PRM) for global path planning.

2.6.2 | Deep Learning Techniques

Navigation systems often rely on specific neural network architectures and learning algorithms to process distinct data inputs, and generate control policies as outputs. Upon critical review on Literature, it is evident that the deep learning approaches of autonomous mobile robot navigation are driven by three architectural paradigms: Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Vision Transformers (ViTs).

Convolutional Neural Networks are the most widely used approach in these systems. This can be attributed to the fact that CNNs are the best fit for the navigation pipeline. CNNs are best suited for processing spatial data such as camera images, depth maps, and lidar projections. The CNN can extract localized information from edges and textures to semantic level concepts, allowing robots to interpret complex environments reliably and efficiently (?). This makes CNNs the best fit for perception tasks such as obstacle detection (???). This makes it the best fit for replacing the classical localization part of the navigation stack.

Recurrent neural networks (RNNs) are widely used in robot navigation due to their ability to model dependencies in sequential data such as robot odometry, velocity history, etc. Unlike CNNs, RNNs with LSTMs, and or GRUs are essential for predicting

sequential robot trajectories, estimating future states, and generating smooth control commands (??). Recent Research incorporates RNNs with reinforcement learning or hybrid deep learning pipelines to better model robot-to-environment interactions. One example is the hybrid RNN architecture proposed by ? which demonstrates an improvement on optimization of path planning and collision avoidance by fusing temporal motion cues with spatial features extracted from CNNs. Although studies also show that that RNN-based models significantly improve localization accuracy by including historical context, reducing hallucinations and localization drift, denoising movements, specifically in environments where GPS is not available (??).

Vision Transformers (ViTs) are a newer deep learning technique that is used in robot navigation, leveraging the breakthroughs in self-attention and the transformer model for vision based tasks. Unlike CNNs, which look at a group of fields at a time, CNNs process the data, its dependencies, and the semantics of each datapoint relative to each other. This helps robots interpret the whole environment, and consequently, plan out safe paths more effectively. (??). Recent studies show that ViTs outperform CNNs in tasks requiring higher level semantic understanding or contextual awareness, such as large scale mapping, multiple map localization, contextual path planning, etc. (?). Their ability to maintain performance under viewpoint changes, lighting variations, and occlusion makes them increasingly relevant for robust real-world navigation.

2.7 | Theoretical Framework

This section outlines the mathematical models and fundamental theories utilized in the development of the autonomous navigation system. Specifically, it discusses the geometric principles of homography for localization, the algorithmic basis of fiducial marker detection, and the architecture of the deep learning approach used in the study.

2.7.1 | Projective Geometry and Planar Homography

Since this study uses a single overhead camera to track a robot on a flat floor, the relationship between the camera's image plane and the real world ground plane is defined by a planar homography. In computer vision, homography is defined as a fundamental projective transformation that maps points from one planar surface to another ?. In our case, given an image point $p = (u, v, 1)^t$ and a corresponding point in the ground-plane coordinate system $p' = (x, y, 1)^t$, the planar homography relates

them as

$$s \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

where

- s is an arbitrary scaling factor,
- H is the 3×3 homography matrix,
- (u, v) are the pixel coordinates of the marker,
- and (x, y) are the metric coordinates on the floor.

To solve for the matrix H , at least four point correspondences are required. This study leverages the Direct Linear Transformation (DLT) algorithm to compute H using calibrated anchors at the corners of the navigation arena, allowing for the real-time conversion of visual data into metric position estimates (?).

2.7.2 | Neural Networks and Convolutional Neural Networks (CNNs)

The control strategy that we proposed in this study is a CNN-based perception to actuation framework to provide end-to-end navigation controls. Unlike classical control methods like PID, which rely on a mathematical method for error correction, this study uses neural networks to approximate the non-linear relationship between the visual input and motor control, effectively emulating the thought process of a human driver. A neural network is a model that aims to imitate the functions of a human brain. It is made up of interconnected nodes (neurons) that are organized into different layers (?). The fundamental operation involves transforming input data x into an input y through a series of weighted sums and non-linear activation functions:

$$h_i = f_i(W_i h_{i-1} + b_i)$$

where h_i is the output of the i -th layer, W_i and b_i are the weight matrix and bias vector, and f_i is the nonlinear activation function.

This study employs a Convolutional Neural Network (CNN) architecture, which is highly effective for processing grid-like data such as images (?). CNNs are characterized by three primary layer types that allow the system to learn spatial hierarchies automatically.

Convolutional Layers. These layers apply a set of kernels that slide over the input image, performing convolution operations to extract spatial features such as edges, textures, etc. The operation is defined as:

$$(K * I)(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n)$$

where I is the input image, K is the kernel, and (i, j) are the spatial coordinates.

Pooling Layers. These layers downsample the feature maps, reducing the dimensionality and the number of parameters. This helps make the features robust to slight shifts or distortions (e.g., Max-Pooling) while lowering the computational load.

Fully Connected Layers. Typically placed at the end of the CNN, these layers flatten the high-level features extracted by the convolutional and pooling layers and map them to the final output. In this study, the output corresponds to the continuous motor power commands (ie. left and right wheel velocities for Ackermann steering).

In the context of this research, CNN functions as an end-to-end policy network. It takes the homography-transformed overhead image, augmented with a digitally rendered coverage path, as input and regresses the optimal motor controls required to track that path. The network is trained using Behavioral Cloning, where the loss function (e.g., Mean Squared Error) minimizes the difference between the network's predicted actions and the human demonstrator's commands during the data collection phase. This approach allows the system to learn complex path-following behaviors directly from visual data, serving as a learned path follower component that bridges the gap between perception and control.

2.7.3 | Imitation Learning and Behavioral Cloning

The study uses Imitation Learning as its deep learning approach. Imitation learning is the industry standard end-to-end approach based on our literature on Chapter 2. Imitation learning assumes access to expert demonstration. The goal is to learn a control policy that mimics the behavior of the expert given specific state observations (?). More specifically, this study implements Behavioral Cloning, the simplest and most

direct form of Imitation learning. Using this, the problem of learning a robot's control policy is then reduced to a supervised learning task. The system collects a dataset D consisting of N pairs of observations and expert actions:

$$D = \{(s_i, a_i)\}_{i=1}^N$$

where s_i represents the state observation (input) at time step i , and a_i represents the expert action (output) at time step i .

Methodology

3.1 | Hardware Setup

3.1.1 | Robot Platform

The mobile agent utilized in this study is a custom-built differential drive robot designed for low-latency execution of remote commands. The platform is intentionally minimized to function as a "remote-controlled" agent, removing the need for heavy onboard computation.

Robot Chassis and Drivetrain

The mobile agent utilizes a DFRobot Devastator Tank Mobile Platform. This platform operates on a 3-8V DC working voltage and features two micro DC geared motors. It weighs 780 g, measures 8.86 x 8.86 x 4.25 inches, and includes driver wheels with a 43mm diameter. The chassis is configured for Differential Drive kinematics, which allows for zero-radius turning and precise heading adjustments.

Embedded Controller

The robot is equipped with a Raspberry Pi 3B running Raspi OS, acting as the low-cost microcontroller and communication bridge. Its sole function is to receive data packets from the central workstation and convert them into Pulse Width Modulation (PWM) signals.

Motor Controller

An L298N H-Bridge DC Motor Controller is used to interface the microcontroller with the DC motors. This provides the necessary current amplification and direction control.

Wireless Handheld Controller

A Logitech Gamepad F710 Joystick Controller with a USB Dongle is utilized for manually controlling the robot during the data collection phase.

Power Supply

The system is powered by a Lithium-Ion battery pack (e.g., 2S or 3S 18650 configuration) to provide stable voltage to the microcontroller and raw power to the motor driver.

3.1.2 | Environment Setup

The experimental environment is a controlled, structured arena designed to implement computer vision processing. A single high-definition RGB webcam (Logitech BRIO) is mounted at a fixed height above the area. The camera is positioned at an angle to cover the entire operational area. While the camera is angled, the software pipeline corrects this perspective distortion mathematically. The arena consists of a flat, non-reflective surface (to minimize glare and specular highlights that could confuse the CNN). The boundaries of the Region of Interest (ROI) are physically defined by four ArUco markers fixed at the corners. These markers serve as the static reference anchors for the system's homography calibration. All image processing, neural network training, and real-time inference are performed on a dedicated laptop or desktop computer equipped with a GPU. This workstation runs the Python-based control software, processes the camera feed via USB, and broadcasts control commands to the robot over the local wireless network.

3.2 | Software Framework

3.2.1 | Core Libraries

- **Python:** The system is designed as a modular, monolithic application that is primarily implemented in the Python programming language.
- **OpenCV:** This library is utilized extensively for the computer vision and perception modules. Specific applications of OpenCV in the system include:
 - Estimating the intrinsic camera parameters using standard checkerboard calibration methods (e.g., the `calibrateCamera` function).
 - Extracting, decoding, and filtering ArUco markers, as well as refining corner coordinates to sub-pixel accuracy using the `cornerSubPix` algorithm.
 - Computing the analytic 3×3 transformation matrix using the `getPerspectiveTransform` function and applying the top-down perspective warp to the raw camera feed via the `warpPerspective` function.
- **PyTorch:** This deep learning library is utilized to construct and execute the Inference Engine (the CNN policy). They are responsible for handling the continuous regression tasks of the modified NVIDIA DAVE-2 Convolutional Neural Network architecture.

3.2.2 | Camera and System Calibration

To ensure the visual input accurately represents the physical world, the system undergoes a two-step calibration process that separates intrinsic camera properties (lens distortion) from the geometric relationship between the camera and the arena (homography).

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \quad (3.1)$$

where:

- (X, Y, Z) are the coordinates of a 3D point in the world coordinate space.
- (u, v) are the coordinates of the projection point in pixels.

- s is a scaling factor.

In practice, wide-angle lenses such as the Logitech BRIO introduce radial and tangential distortion (?). These are modelled by the following correction terms:

Radial Distortion:

$$x_{corrected} = x(1 + k_1r^2 + k_2r^4 + k_3r^6) \quad (3.2)$$

$$y_{corrected} = y(1 + k_1r^2 + k_2r^4 + k_3r^6) \quad (3.3)$$

Tangential Distortion

$$x_{corrected} = x + [2p_1xy + p_2(r^2 + 2x^2)] \quad (3.4)$$

$$y_{corrected} = y + [p_1(r^2 + 2y^2) + 2p_2xy] \quad (3.5)$$

where:

- k_1, k_2, k_3 are radial coefficients,
- p_1, p_2 are tangential coefficients and
- $r^2 = x^2 + y^2$

In OpenCV (?), these coefficients are estimated once using the `calibrateCamera` function via standard checkerboard calibration (?), and stored for all subsequent image correction operations. The Levenberg–Marquardt algorithm (?) minimizes the reprojection error between detected and projected checkerboard corners to produce the optimal intrinsic matrix K and distortion vector.

3.2.3 | Perception and Localization Module

The localization module operates in two phases: a one-time **Initialization Phase** that detects the static arena boundaries and computes the homography matrix, and a continuous **Runtime Phase** that applies this mapping to warp each incoming frame for the control policy. These phases are formalized in Algorithm 1.

3.2.4 | ArUco Marker Detection

The localization pipeline begins with the detection of ArUco markers (Garrido-Jurado et al., 2014), to define the Area of Interest (AOI). We utilize the `DICT_5X5_100` dictionary (5x5 bit grid, 100 unique IDs), which offers a balance between decoding speed and false-positive rejection.

Marker Configuration and Corner Assignment

To resolve the ambiguity of the arena's orientation and prevent the coordinate system from mirroring or rotating if the camera placement changes, specific unique IDs are assigned to the four physical corners of the operational area. This fixed assignment acts as a ground truth for the homography estimation. The configuration is explicitly defined as follows:

1. Top-Left (TL): Marker ID 0
2. Top-Right (TR): Marker ID 1
3. Bottom-Right (BR): Marker ID 2
4. Bottom-Left (BL): Marker ID 3

Detection is performed on the undistorted frame — not the raw frame — to ensure that the resulting homography matrix is free of lens distortion artifacts. The detection pipeline proceeds as follows:

1. **Preprocessing:** The RGB frame is converted to grayscale.
2. **Candidate Extraction:** Adaptive thresholding segments black borders; contours are filtered by convexity and square geometry.
3. **Bit Decoding:** The inner region of each candidate is divided into a grid and its binary pattern is decoded to recover the marker ID.
4. **Corner Refinement:** Corner positions are refined to sub-pixel accuracy using `cornerSubPix`.

The output is a set of four ordered corner coordinates from the undistorted image. It should be noted that these markers are not attached to the robot, they serve only to anchor the global coordinate system of the arena for the homography calculation.

3.2.5 | Planar Homography Estimation

The geometric relationship between the camera's image plane and the physical floor is modelled as a Planar Homography (?), an invertible projective transformation mapping image pixel coordinates $p = [u, v, 1]^T$ to world coordinates $P_w = [x, y, 1]^T$:

$$s[u, v, 1]^T = H [u, v, 1]^T \quad (3.6)$$

where H is a 3×3 matrix with 8 degrees of freedom. Given the four sorted ArUco corner correspondences from Section 3.2.4, H is solved analytically via the Direct Linear Transformation (DLT) algorithm (?) using OpenCV's `getPerspectiveTransform` function.

Perspective Warping (Runtime Application). During runtime, the pre-computed H is applied to every incoming frame via `warpPerspective`:

$$\text{Img}_{world} = \text{warpPerspective}(\text{Img}_{raw}, H) \quad (3.7)$$

This produces a rectified Bird's-Eye View in which the four corner markers align exactly with the image boundaries, giving the CNN a geometrically stable input that is invariant to minor shifts in camera placement or angle.

3.3 | System Integration and Control Flow

The proposed system implements a centralized, vision-based autonomous navigation architecture that offloads all image processing and CNN inference to a remote workstation, communicating control commands to the robot over a local wireless link. The full architectural data flow is illustrated in Figure 3.1.

As depicted in Figure 3.1, the system architecture is divided into three functional subsystems: sensing, processing, and actuation. Together, these components form a closed-loop control pipeline for autonomous navigation.

- **Sensing Subsystem (Arena):** Located in the physical environment, this block consists of an overhead camera and fixed ArUco markers. The camera captures the entire operational arena and provides the raw visual input used by the system.

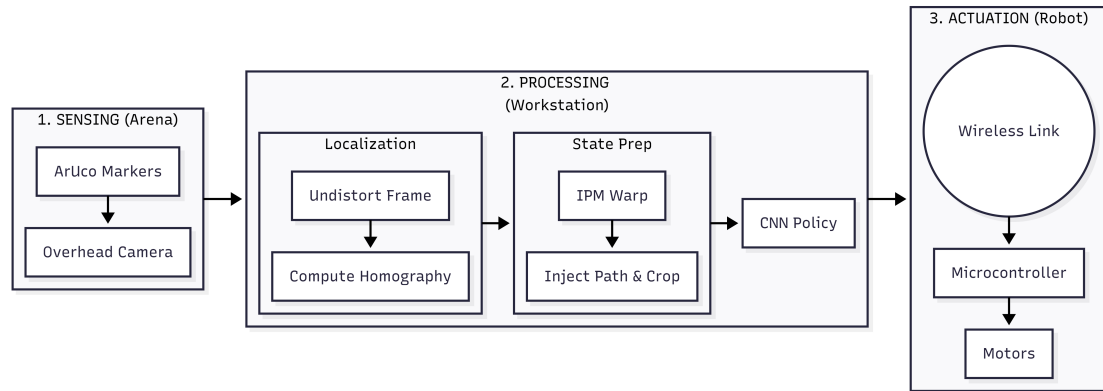


Figure 3.1: System Architecture.

The ArUco markers act as reference points for geometric correction and spatial alignment.

■ **Processing Subsystem (Workstation):** This block contains the core intelligence of the system and executes a two-stage processing pipeline:

- **Localization and State Preparation:** The system detects ArUco markers to compute a homography matrix, which warps the angled camera feed into a standardized top-down bird’s-eye view using inverse perspective mapping (IPM). The transformed image is cropped and augmented with the target navigation path to form the system state.
- **CNN-Based Policy Inference:** The pre-processed visual state is fed directly into a convolutional neural network (CNN), which serves as an end-to-end behavioral policy mapping visual observations directly to low-level motor control commands.

■ **Actuation Subsystem (Robot):** The inferred control commands, including steering and throttle signals, are transmitted wirelessly via Wi-Fi or Bluetooth to the mobile robot. The robot’s onboard microcontroller receives these commands and drives the motors, completing the closed-loop control cycle.

3.3.1 | Module Integration

The software is organized into four distinct functional modules, ensuring separation of concerns and allowing independent testing of subsystems.

1. **Perception Interface (*CameraStream*):** Interfaces with the hardware camera driver and manages the frame buffer to ensure the processing pipeline always retrieves the most recent frame, minimizing input lag.
2. **State Estimator (*Preprocessor*):** Encapsulates the pre-computed homography matrix H from Section 3.2.5 and executes the full state representation pipeline — perspective warp, semantic extraction, path injection, egocentric crop, and normalization.
3. **Inference Engine (*PilotModel*):** Loads the pre-trained CNN weights into memory on startup and exposes a `predict()` method that accepts a processed image tensor and returns a normalized steering command.
4. **Actuation Interface (*MotorDriver*):**
 - Abstracts low-level GPIO commands from the rest of the pipeline.
 - Converts the normalized CNN output into a raw steering command using a fixed gain and saturation, then maps it to hardware-specific PWM duty cycles transmitted to the L298N motor driver.
 - Triggers a safety stop, defaulting all motors to zero velocity if the control loop is interrupted or the watchdog timer expires.

Algorithm 1 Homography Estimation and Perspective Warping

```

1: function INITIALIZEANDWARP(raw_frame, camera_matrix, dist_coeffs, arena_dims)
2:   Input:
3:     raw_frame — Raw RGB image from the camera
4:     camera_matrix(K), dist_coeffs(D) — Intrinsic calibration parameters
5:     arena_dims — Target width and height of bird's-eye view
6:   Output: warped_frame, homography_matrix

7:   // Phase 1: Undistortion
8:   undistorted_frame  $\leftarrow$  UNDISTORT(raw_frame, K, D)
9:   gray_frame  $\leftarrow$  CONVERTTOGRAY(undistorted_frame)

10:  // Phase 2: Marker Detection
11:  dictionary  $\leftarrow$  GETDICTIONARY(DICT_4X4_50)
12:  (corners, ids)  $\leftarrow$  DETECTMARKERS(gray_frame, dictionary)

13:  // Phase 3: ID-Based Sorting (Ground Truth Assignment)
14:  // Marker IDs: 0=TL, 1=TR, 2=BR, 3=BL
15:  expected_ids  $\leftarrow$  {0, 1, 2, 3}
16:  if |ids| < 4 or expected_ids  $\not\subseteq$  ids then
17:    return Error "Markers 0–3 not found"
18:  end if
19:  src_points  $\leftarrow$  empty list of size 4
20:  for i  $\leftarrow$  1 to |ids| do
21:    if ids[i] = 0 then
22:      src_points[0]  $\leftarrow$  corners[i] ▷ Top-Left
23:    else if ids[i] = 1 then
24:      src_points[1]  $\leftarrow$  corners[i] ▷ Top-Right
25:    else if ids[i] = 2 then
26:      src_points[2]  $\leftarrow$  corners[i] ▷ Bottom-Right
27:    else if ids[i] = 3 then
28:      src_points[3]  $\leftarrow$  corners[i] ▷ Bottom-Left
29:    end if
30:  end for

31:  // Phase 4: Homography Calculation
32:  dst_points  $\leftarrow$  {(0,0), (W,0), (W,H), (0,H)}
33:  homography_matrix  $\leftarrow$  GETPERSPECTIVETRANSFORM(src_points, dst_points)

34:  // Phase 5: Perspective Warping
35:  warped_frame  $\leftarrow$  WARPERSPECTIVE(undistorted_frame, homography_matrix, arena_dims)
36:  return warped_frame, homography_matrix
37: end function

```

Algorithm 2 Navigation Agent: Neural Network Inference

```

1: function COMPUTECONTROLVECTOR(warped_frame, cnn_model)
2:   Input: Top-down image and pre-trained CNN model
3:   Output: Steering angle and throttle commands

4:   // Step 1: Pre-processing
5:   input  $\leftarrow$  RESIZE(warped_frame,  $200 \times 200$ )
6:   input  $\leftarrow$  NORMALIZEPIXELVALUES(input) ▷ Scale to range [0, 1]

7:   // Step 2: Deep Learning Inference
8:   (raw_steer, raw_throttle)  $\leftarrow$  MODEL PREDICT(cnn_model, input)
9:   ksteer  $\leftarrow$  50 ▷ Scaling gain from normalized model output to raw command
10:  steering_cmd  $\leftarrow$  ksteer · raw_steer ▷ Scale normalized steering
11:  steer_min, steer_max  $\leftarrow$  -50, 50 ▷ Raw command saturation bounds

12:  // Step 3: Safety Saturation
13:  ▷ Clip outputs to ensure values are within hardware limits
14:  steering  $\leftarrow$  CLIP(steering_cmd, steer_min, steer_max)
15:  throttle  $\leftarrow$  CLIP(raw_throttle, 0.0, 1.0)
16:  return steering, throttle
17: end function

```

Algorithm 3 Actuation: Hardware Control Interface

```

1: function DRIVEROBOT(steering, throttle)
2:   Input: Normalized control commands from Neural Network
3:   Output: Physical electrical signals to motors

4:   // Step 1: Safety Check
5:   if Watchdog Timer Expired then
6:     TRIGGEREMERGENCYSTOP()
7:     return
8:   end if

9:   // Step 2: Signal Conversion
10:  ▷ Convert normalized values to Pulse Width Modulation (PWM)
11:  servo_signal  $\leftarrow$  MAPTOSERVODUTYCYCLE(steering)
12:  motor_signal  $\leftarrow$  MAPTODCMOTORSPEED(throttle)

13:  // Step 3: Hardware Execution
14:  WRITEGPIO(SERVO_PIN, servo_signal)
15:  WRITEGPIO(MOTOR_PIN, motor_signal)
16: end function

```

3.3.2 | Real-Time Execution and Middleware

The system operates using a synchronous main control loop. Unlike asynchronous systems that rely on complex middleware (e.g., ROS) for message passing, this study utilizes a direct memory access approach to minimize overhead and latency. This design choice is critical for achieving stable end-to-end controller without PID dampening.

Execution Flow The main execution pipeline follows a “Sense-Think-Act” cycle:

1. **Initialization Phase:** On startup, the system verifies hardware connections, warms up the camera sensor (allowing auto-exposure to settle), and loads the CNN model into RAM.
2. **The Control Loop:** The system enters a `while(true)` loop that executes the following sequence:
 - **Acquire:** The `CameraStream` fetches the latest frame.
 - **Process:** The `Preprocessor` warps and crops the image.
 - **Infer:** The `PilotModel` computes the control vector.
 - **Drive:** The `MotorDriver` executes the command.
 - **Log:** (Optional) Data is saved for debugging or further training.

Timing and Latency To maintain smooth motion, the software is designed to run at a target frequency of 15Hz to 20Hz. The most computationally expensive step is the CNN inference; therefore, the input image resolution is kept minimal to mitigate latency. During physical deployment the recorded telemetry timestamps indicate an effective sampling/inference frequency of approximately 60 Hz; Chapter 4 reports per-frame durations using this observed rate where relevant. A software watchdog is implemented to monitor the loop time; if the camera disconnects or the loop hangs, the actuation interface triggers a safety stop to prevent runaway behavior.

3.4 | Data Acquisition and Processing

With the perception and localization pipeline established in the preceding sections, the system transitions from passive environment sensing to active knowledge

acquisition. This phase operationalizes the Behavioral Cloning paradigm described in Section 2.7.3, constructing the supervised dataset that will serve as the sole source of driving knowledge for the Convolutional Neural Network. The data acquisition pipeline is designed to ensure that every recorded sample is a semantically meaningful, temporally synchronized pair of a preprocessed visual state and a human-demonstrated steering command. Telemetry frames are recorded frame-by-frame during deployment; the recorded timestamps show an effective sampling frequency of roughly 60 Hz.

3.4.1 | Dataset Collection

The dataset $D = \{(s_i, a_i)\}_{i=1}^n$ is generated through a series of manual driving sessions in which a human operator controls the differential-drive robot using a gamepad controller while the overhead camera system and data logger run concurrently in real time.

3.4.1.1 | Teleoperation Setup

During collection sessions, the robot is driven remotely via a gamepad (e.g., Xbox-compatible controller) interfaced with the workstation through Python’s `pygame` library. The left analog stick’s horizontal axis is mapped to a normalized steering command $\hat{u}_t \in [-1.0, 1.0]$ and then scaled by a fixed steering gain to the raw command a_t (we use $k_{steer} = 50$, so $a_t = k_{steer} \cdot \hat{u}_t$, yielding raw command units in approximately $[-50, 50]$). Negative values correspond to left turns, positive values to right turns, and $a_t \approx 0$ indicates straight-line motion. Throttle is held at a fixed, conservative value throughout data collection, consistent with the deployment strategy described in Section 3.5.4, so that the network is trained exclusively as a steering regressor.

3.4.1.2 | Synchronization Protocol

Temporal alignment between the visual state and the control action is the most critical requirement for producing a valid behavioral cloning dataset. A mismatch of even a single frame can corrupt the steering label associated with a given visual observation, introducing noise that degrades the learned policy. To eliminate this ambiguity, a dedicated data logger script executes within the same synchronous control loop described in Section 3.6.2. On every iteration of the loop, the following operations are performed atomically before any data is written to disk:

1. The latest undistorted frame is fetched from the `CameraStream` buffer.
2. The `Preprocessor` module applies the full state representation pipeline (detailed in Section 3.4.2) to produce the final input tensor s_t .
3. The current gamepad axis reading is polled via `pygame.event.pump()` to obtain the instantaneous steering value a_t .
4. The pair (s_t, a_t) is appended to a running log as a NumPy array and the corresponding raw steering float is recorded to a paired CSV file, keyed by a shared frame index.

This within-loop atomicity guarantees that every saved image corresponds to the exact steering command the operator applied at that moment, with no buffering delay between visual capture and control sampling.

3.4.1.3 | Dataset Balancing

A naïve collection strategy produces a dataset heavily biased toward straight-line driving, since the majority of any track consists of straight segments. A model trained on such an imbalanced distribution will learn to output near-zero steering commands by default, a degenerate policy that performs poorly on turns and recovery scenarios. To counteract this, the dataset is deliberately structured across three driving modes:

- **Nominal Laps:** The operator drives the robot smoothly along the digitally injected reference path, covering straight segments, left turns, right turns, and the transitions between them.
- **Aggressive Turn Demonstrations:** Additional dedicated runs are recorded in which the operator repeatedly navigates sharp 90-degree and U-turn topologies at the boundary of the robot's turning radius, increasing the representation of high-magnitude steering commands.
- **Recovery Maneuvers:** The robot is intentionally displaced laterally from the center of the reference path and then corrected back by the operator. These samples are critical for generalization: they teach the network the corrective steering behavior required to recover from covariate shift during autonomous deployment, a known failure mode of pure behavioral cloning ?.

After collection, the dataset is post-processed to enforce approximate class balance across steering angle bins. Frames corresponding to near-zero steering ($|a_t| < 0.05$) are subsampled to ensure they do not dominate the training distribution, while recovery and high-curvature samples are retained in full.

3.4.2 | State Representation

Before any recorded frame is passed to the neural network, either during training or inference, it has to go through a deterministic, multi-stage preprocessing pipeline that transforms the raw overhead camera output into a compact, geometrically meaningful state representation. The goal of this pipeline is to enforce a strict information bottleneck: the network must receive only the visual features that are causally relevant to the steering decision, with all spurious environmental correlates which include floor texture, ambient shadow, arena boundary color, and absolute spatial position. In this pipeline, it is then systematically eliminated.

The complete pipeline proceeds through the following stages.

3.4.2.1 | Stage 1 – Homography Warp (Bird's-Eye View Rectification)

The raw, undistorted frame from the overhead camera is transformed using the pre-computed homography matrix H (computed during the initialization phase described in Section 3.4.2) via OpenCV's `warpPerspective` function:

$$\text{Img}_{\text{world}} = \text{warpPerspective}(\text{Img}_{\text{raw}}, H, \text{arena_dims}) \quad (3.8)$$

This produces a geometrically rectified, orthographic top-down view of the arena in which pixel distances correspond to fixed metric distances on the floor, and the four ArUco corner markers align exactly with the image boundaries. The output is a standardized $W \times H$ pixel canvas representing the full area of interest.

3.4.2.2 | Stage 2 – Semantic Polygon Extraction (Black Canvas Rendering)

To decouple the network from the physical appearance of the arena floor, the drivable track boundaries within the warped image are isolated and re-rendered as discrete geometric polygons on a pure black canvas. The track boundaries are detected as contiguous regions of a known color or intensity within the warped frame and extracted using OpenCV's contour detection pipeline. The resulting polygons are

drawn in white onto a zero-valued (black) background image of identical dimensions. This semantic abstraction ensures the following:

- All RGB texture, floor reflections, and lighting variation are discarded.
- The network receives only the binary geometry of the drivable surface — the features that are invariant across sessions, lighting conditions, and track configurations.
- The model no longer processes an image of “a robot on a floor” but rather a normalized geometric representation of path curvature relative to the robot’s position.

3.4.2.3 | Stage 3 — Path Injection (State Augmentation)

The semantic black canvas alone conveys the shape of the drivable area but does not communicate the robot’s navigation objective — the specific trajectory it is expected to follow within that area. To give the network directional intent, the target reference path is digitally rendered onto the black canvas as a second visual channel. The path is represented as a polyline or filled polygon, drawn in a distinct intensity value, that traces the desired centerline through the current track topology. The resulting composite image contains exactly two classes of information: the track boundaries (giving the network the constraint space) and the reference path (giving it the optimization target).

3.4.2.4 | Stage 4 — Egocentric Dynamic Cropping (ROI Extraction)

As established in the preliminary evaluation (Section 4.2), feeding the full warped arena into the CNN causes the network to overfit to the robot’s absolute pixel coordinates rather than learning generalizable path geometry. The definitive solution is the egocentric dynamic crop: at every time step, the robot’s current pixel centroid (c_x, c_y) is detected from the chassis-mounted ArUco marker within the warped frame, and a fixed-dimension bounding window of size $W_{crop} \times H_{crop}$ (e.g., 200×200 pixels) is extracted centered on that coordinate:

$$s_t = \text{canvas} \left[c_y - \frac{H_{crop}}{2} : c_y + \frac{H_{crop}}{2}, c_x - \frac{W_{crop}}{2} : c_x + \frac{W_{crop}}{2} \right] \quad (3.9)$$

By continuously re-centering the crop on the robot, its absolute position within the global arena is rendered invariant. From the network’s perspective, the robot does not move; instead, the semantically extracted path geometry translates and rotates around the robot’s fixed origin. The only dynamic variables passed to the convolutional feature extractor are the relative orientation, curvature, and proximity of the approaching path boundaries — the precise features required to generate a geometrically correct steering command.

3.4.2.5 | Stage 5 — Normalization

The cropped binary canvas is cast to a 32-bit float array and normalized to the range $[0.0, 1.0]$ by dividing by 255. This normalization is consistent with the input normalization layer of the DAVE-2 architecture (Table 3.1), which further linearly scales pixel values to $[-1.0, 1.0]$ via the expression:

$$x_{norm} = \frac{x}{127.5} - 1 \quad (3.10)$$

Normalizing prior to storage reduces numerical variance across the dataset and accelerates gradient descent convergence during training. The output of Stage 5 is the final state tensor s_t — a $W_{crop} \times H_{crop} \times 1$ grayscale array — that is saved to disk paired with its corresponding steering label a_t during data collection, and fed directly into the CNN’s input layer during both training and deployment inference.

3.5 | Convolutional Neural Network Model and Training

This section details the implementation of the deep learning pipeline, translating the theoretical framework of Behavioral Cloning and CNNs (discussed in Section 2.7) into a functional navigation system. The pipeline is divided into model architecture design, data preprocessing, acquisition, training, and final deployment.

3.5.1 | Model Architecture

The study employs a modernized Convolutional Neural Network (CNN) architecture based on the NVIDIA DAVE-2 model, optimized specifically for continuous regression tasks. Unlike classification networks that output probabilities for discrete categories,

this network is designed to output continuous control values. The architecture follows a standard "encoder-decoder" style flow adapted for control, starting with an Input Layer that accepts the preprocessed image tensors (dimensions defined in Section 3.6.2).

The core processing begins with the Convolutional Feature Extractor, which utilizes five sequential convolutional layers to extract spatial features from the homography-transformed overhead view. The first three layers utilize 5x5 kernels with a stride of 2 to rapidly reduce spatial dimensions and capture broad geometric structures, such as the path boundaries. The subsequent two layers employ 3x3 kernels with a stride of 1 to capture finer, higher-level spatial hierarchies. To mitigate the vanishing gradient problem and accelerate convergence, every convolutional layer is followed by Batch Normalization and an Exponential Linear Unit (ELU) activation function, which pushes mean activations closer to zero compared to standard ReLU.

Following feature extraction, a flattening operation converts the multidimensional feature maps into a 1D vector (containing 20,736 elements for a 200x200 input), preparing the data for high-level reasoning.

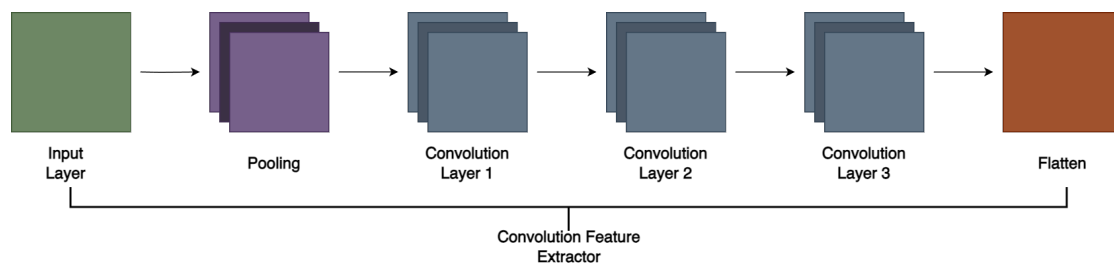


Figure 3.2: Feature Extraction Block: Input, Convolutional Layers, and Flattening

Once flattened, the data passes through the Fully Connected (Dense) Layers. Three hidden dense layers (containing 100, 50, and 10 neurons, respectively) interpret the extracted spatial features to serve as the decision-making bottleneck. These layers also utilize Batch Normalization and ELU activations. Furthermore, a mild Dropout rate of 10% (0.1) is applied to the first two dense layers to provide regularization and prevent overfitting, specifically tuned to avoid variance shifts when combined with Batch Normalization. The pipeline concludes with the Output Layer, consisting of a single neuron utilizing a Hyperbolic Tangent (Tanh) activation function. This bounds the normalized steering output to $[-1.0, 1.0]$, which is then scaled to the raw steering command for actuation and logging.

Layer	Type	Filters / Neurons	Kernel Size	Stride	Activation	Regularization
Input	Normalization	–	–	–	$(x/127.5) - 1$	–
1	Convolutional	24	5×5	2	ELU	Batch Norm
2	Convolutional	36	5×5	2	ELU	Batch Norm
3	Convolutional	48	5×5	2	ELU	Batch Norm
4	Convolutional	64	3×3	1	ELU	Batch Norm
5	Convolutional	64	3×3	1	ELU	Batch Norm
6	Flatten	20,736	–	–	–	–
7	Dense (FC)	100	–	–	ELU	Batch Norm, Dropout (0.1)
8	Dense (FC)	50	–	–	ELU	Batch Norm, Dropout (0.1)
9	Dense (FC)	10	–	–	ELU	Batch Norm
10	Dense (Output)	1	–	–	Tanh	–

Table 3.1: Neural Network Architecture

3.5.2 | Training Configuration

The network is trained as a supervised regression task using the behavioral cloning dataset constructed in Section 4 (Data Acquisition and Processing). The training pipeline is structured in two phases.

Phase 1: Dataset Preparation

Prior to training, the collected dataset undergoes a final preparation step. The full dataset is partitioned using an 80:20 split, with 80% of samples reserved for training and 20% held out for validation. To prevent the training loop from processing samples in a fixed sequence, which could introduce temporal autocorrelation artifacts, the training split is shuffled before each epoch. A batch size of 32 is used throughout training.

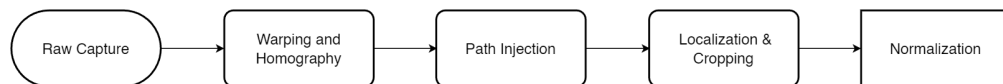


Figure 3.3: The image preprocessing pipeline converting raw camera input into a state representation for the CNN

Phase 2: Network Optimization

The optimization objective is to minimize the Mean Squared Error (MSE) between the predicted steering command and the human demonstrator's recorded command across all samples in each batch:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right)^2 \quad (3.11)$$

The Adam optimizer is selected for its adaptive learning rate capabilities, ensuring stable convergence despite the non-convex nature of the loss landscape. Training proceeds until the validation loss plateaus, at which point Early Stopping is triggered to prevent overfitting and to restore the weights from the best-performing epoch.

The throttle command is held at a fixed, conservative value during both data collection and inference rather than being predicted as a second CNN output. This design decision ensures training stability: in behavioral cloning, predicting two continuous outputs requires the MSE loss to simultaneously balance two potentially conflicting gradient signals during backpropagation. Restricting the network to steering-only prediction provides a cleaner, more interpretable loss signal and allows the model to converge more efficiently. This approach also follows the precedent established by NVIDIA's PilotNet architecture, which originally focused exclusively on steering angle prediction with throttle managed as an independent system.

3.5.3 | Deployment and Inference

The trained model is exported and optimized for the target hardware. The deployment process operates in a continuous control loop.

1. **Model Serialization:** The trained network architecture and weights are saved in a portable format (e.g., .h5 or .tflite).
2. **Inference Loop:** The deployment script runs on the central processing unit (CPU) or the robot's onboard computer in a continuous loop:
 - a. Capture and warp the overhead camera image using the homography transformation.
 - b. Preprocess the image and extract the Region of Interest (ROI).
 - c. Feed the processed sensor input into the Convolutional Neural Network (CNN).
 - d. The CNN outputs raw regression values corresponding to control commands (e.g., steering = 0.3, throttle = 0.8).

- e. The predicted values are scaled to the appropriate PWM range (0–255) and transmitted directly to the motor drivers via GPIO, bypassing any PID or additional error-correction logic.

3.6 | Evaluation and Testing

To rigorously assess the performance of the end-to-end CNN controller, a series of controlled experiments were designed. These tests aim to quantify the system’s ability to track a digitally rendered path using only visual inputs, specifically evaluating whether the network has learned generalizable kinematic behaviors or merely memorized spatial coordinates. This section outlines the testing parameters, the track topologies, and the mathematical metrics used to validate the system.

3.6.1 | Physical Testing Environment and Setup

Because the preprocessing pipeline systematically eliminates ambient RGB textures, shadows, and background noise, the physical testing does not require a visually isolated robotics arena. Testing was conducted on a standard, flat indoor surface with consistent artificial lighting. The system was deployed following the environment setup as defined in Section ??.

3.6.2 | Evaluation Scenarios

The system was tested across three track configurations of increasing Bézier curve complexity, each designed to isolate specific kinematic behaviors and evaluate the CNN policy’s adaptability. All tests were conducted in a controlled indoor environment with constant artificial lighting to minimize perception noise.

1. **Scenario 1 (45-Degree Bézier Curves):** This topology features mild, continuous curvature to assess the model’s baseline tracking stability. The primary goal of this test is to evaluate whether the CNN can generate smooth, continuous steering adjustments under gradual geometric changes without exhibiting oscillatory steering behavior (fishtailing) or overreacting to shallow curves.
2. **Scenario 2 (90-Degree Bézier Corners/Square Path):** This scenario tests the model’s response to rapid, orthogonal geometric transitions. Navigating a square path with rounded corners evaluates the network’s ability to map sudden

geometric shifts to aggressive, high-magnitude steering commands and rapidly recover heading and straight-line stability upon corner exit.

3. **Scenario 3 (180-Degree Sustained U-Turns):** This loop features extreme, continuous curvature to test the system’s kinematic endurance under prolonged geometric distortion. The scenario evaluates the network’s ability to scale and sustain near-maximum steering angles for extended durations, and smoothly transition through extreme curve sections.

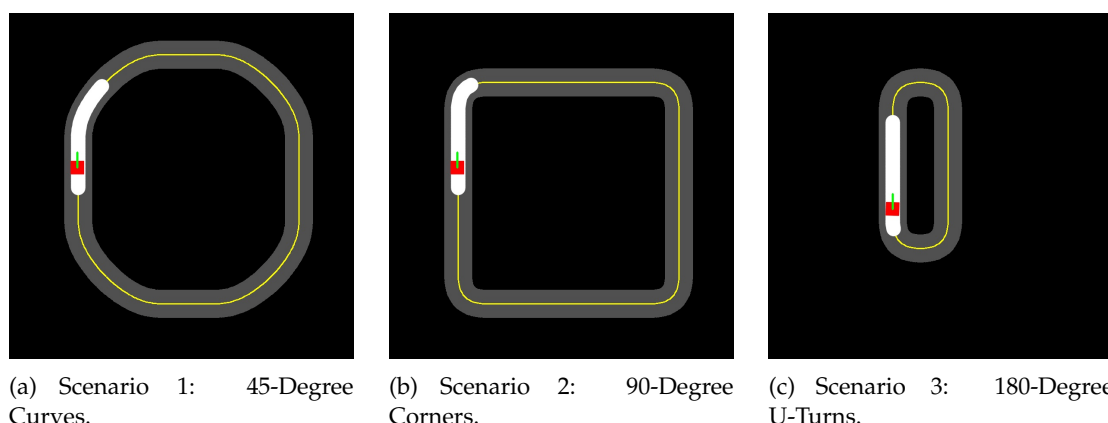


Figure 3.4: Visual representation of the three experimental track configurations used for physical testing.

3.6.3 | Ground-Truth Tracking System

To accurately measure the real-world kinematic performance of the robot without relying on manual estimation, the system utilizes an automated error-logging script during physical deployment. By tracking the robot’s chassis-mounted ArUco marker relative to the digitally injected reference path, the script directly computes instantaneous error metrics frame-by-frame; the recorded timestamps show an effective sampling frequency of approximately 60 Hz during deployment.

Unlike traditional SLAM-based evaluations that require logging global (X, Y) spatial coordinates and absolute yaw angles, this lightweight evaluation pipeline records only the direct relative deviations between the robot and the reference path. This approach minimizes data overhead while providing a precise, continuous measurement of the controller’s tracking accuracy throughout each trial.

3.6.4 | Performance Metrics

The evaluation metrics are divided into two distinct categories: theoretical metrics utilized during the offline supervised learning phase, and real-world kinematic metrics utilized during physical deployment.

3.6.4.1 | Theoretical Metrics

These metrics are calculated exclusively during the offline training phase of the CNN. They measure the mathematical variance between the human-provided steering angles from the behavioral cloning dataset and the model's predicted steering angles.

The **Mean Squared Error (MSE)** serves as the primary loss function during backpropagation. It heavily penalizes large deviations between predicted and actual steering values:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \left(y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right)^2 \quad (3.12)$$

The **Mean Absolute Error (MAE)** provides a linear, easily interpretable measure of the average steering angle error in normalized units during the validation phase:

$$\mathcal{L}_{MAE} = \frac{1}{N} \sum_{i=1}^N \left| y_{\text{pred}}^{(i)} - y_{\text{true}}^{(i)} \right| \quad (3.13)$$

3.6.4.2 | Real-World Kinematic Metrics

Because theoretical metrics can be deceptive due to factors such as covariate shift or spurious background correlations, the ultimate validation of the system relies on physical deployment metrics. These measure the actual spatial performance of the differential-drive robot on the floor across all three evaluation scenarios defined in Section 3.6.2.

Cross-Track Error (CTE) and RMSE. The fundamental measure of tracking accuracy is the Cross-Track Error (CTE), defined as the signed perpendicular distance between the robot's geometric center and the nearest point on the digital reference path at any given time step t . Formally, given the robot's center (c_x, c_y) , the nearest point on the path (p_x, p_y) , and the unit path tangent vector (t_x, t_y) at that point, the signed CTE is computed as:

$$CTE_t = \frac{t_x(c_y - p_y) - t_y(c_x - p_x)}{\sqrt{t_x^2 + t_y^2}} \quad (3.14)$$

where:

- (c_x, c_y) is the robot's geometric center in warped-frame pixel coordinates,
- (p_x, p_y) is the nearest point on the digital reference path,
- (t_x, t_y) is the unit tangent vector of the path at that nearest point.

A positive value indicates the robot is to the right of the path direction; a negative value indicates it is to the left. To provide a single summary statistic for an entire run, the Root Mean Square Error (RMSE) of the per-frame CTE deviations is calculated as:

$$RMSE = \sqrt{\frac{1}{N} \sum_{t=1}^N CTE_t^2} \quad (3.15)$$

A lower RMSE indicates tighter adherence to the reference path. CTE is measured in warped-frame pixel units, which are geometrically consistent across all arena configurations since the homography transformation always maps the arena to a fixed output resolution regardless of physical area dimensions.

Heading Error. Complementing the position error is the Heading Error, which evaluates the robot's orientation accuracy. This metric measures the angular difference between the robot's current yaw angle θ_{robot} , derived from the ArUco marker's top-edge midpoint vector, and the tangent angle of the path θ_{path} at the nearest projected point:

$$\psi_t = \theta_{robot} - \theta_{path} \quad (3.16)$$

The result is wrapped to the range $[-\pi, \pi]$ to ensure sign consistency, and is reported in degrees for interpretability. Monitoring heading error is critical for predictive analysis, as high angular deviations often precede a complete departure from the lane even when the instantaneous CTE remains low.

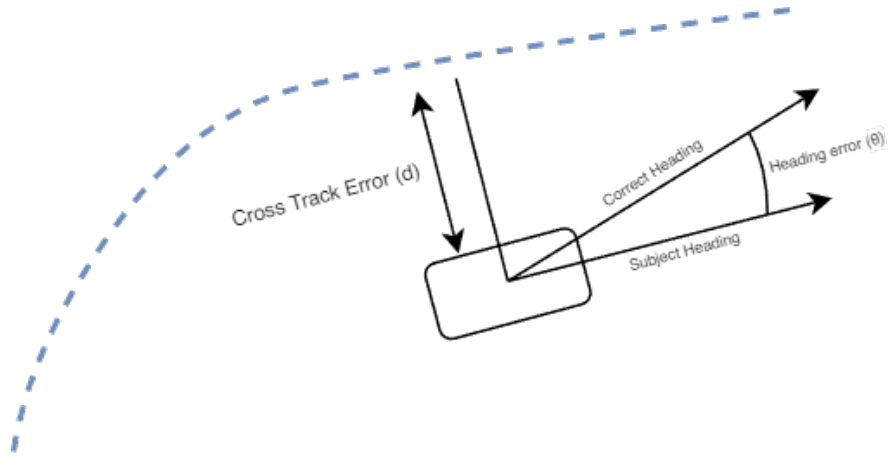


Figure 3.5: Visual representation of Cross-Track Error (d) and Heading Error (θ).

Success Rate. Finally, the overall reliability of the system is measured by the **Success Rate**, defined as the percentage of total attempts in which the robot completed the entire course without any part of its chassis crossing the lane boundaries:

$$\text{Success Rate} = \frac{\text{Number of Successful Laps}}{\text{Total Attempts}} \times 100\% \quad (3.17)$$

A lap is considered failed when the chassis-mounted ArUco marker is no longer detected within the drivable region for more than 30 consecutive frames, corresponding to approximately 1.5 seconds at the system's operating frequency of 15–20 Hz.

Results & Discussion

This chapter presents the evaluation of the developed end-to-end autonomous navigation system. The results are presented chronologically, tracing the system's iterative development. All physical deployment and inference runs reported in this chapter were executed on the real robot, with the trained model running onboard a Raspberry Pi embedded on the platform.

To validate the necessity of the final image preprocessing architecture, the chapter first details the offline training and real-world physical testing of two early experimental baselines. By analyzing the specific failure modes of these early iterations, the empirical justification for the proposed Egocentric Dynamic Crop Model is established.

4.1 | Chronological Development and Experimental Baselines

During the development of the navigation framework, the state representation of the track underwent three major iterations. All models were trained using a modified NVIDIA DAVE-2 Convolutional Neural Network (CNN) architecture with 2,211,175 parameters, processing 200×200 pixel images, and minimizing Mean Squared Error (MSE) using the Adam optimizer.

4.1.1 | Iteration 1: Baseline A (Allocentric Full AOI Model)

The initial approach utilized the raw, top-down Bird’s-Eye View (BEV) image, retaining the room’s floor textures and permanent background elements.

During offline training, Baseline A converged rapidly. As shown in Figure 4.1, both MSE and MAE decreased exponentially, with validation loss plateauing near 0.05 and 0.13, and reaching a low validation MSE of 0.0489 by Epoch 30.

However, physical deployment resulted in immediate, catastrophic navigation failure. On the physical track, the robot exhibited erratic motor control, immediately veering off-path and breaching the arena boundaries.

As shown in Figure 4.2, both the Cross-Track Error (CTE) and the Heading Error (θ_{err}) diverged immediately on deployment (CTE quickly exceeding -100 pixels and Heading Error dropping past -60°). Baseline A maintained the path for only 636 frames with an RMSE of 30.9992 pixels. When analyzing the Error Compounding Rate over the first 500 frames, Baseline A showed rapid, exponential error growth. Minor starting position changes on the real track confused the network, causing it to immediately steer off course.

Post-deployment analysis revealed the model suffered from **shortcut learning**. Because the room environment and global track remained static in the camera’s view, the CNN bypassed learning the path geometry entirely. Instead, it overfit directly to the robot’s absolute pixel coordinates within the room frame. The validation error remained low during training because validation images came from that same static frame. During inference, any minor starting deviation created an out-of-distribution pixel state that the model could not interpret.

4.1.2 | Iteration 2: Baseline B (Feature-Extracted Allocentric Model)

To eliminate background shortcuts like floor reflections, shadows, and pixel discrepancies, the pipeline was updated to add a semantic polygon extraction step. This process isolated the AOI as flat white shapes with a red polygon on a pure black canvas, removing all environmental noise and forcing the network to learn the path geometry itself.

While this removed environmental noise, offline training metrics immediately

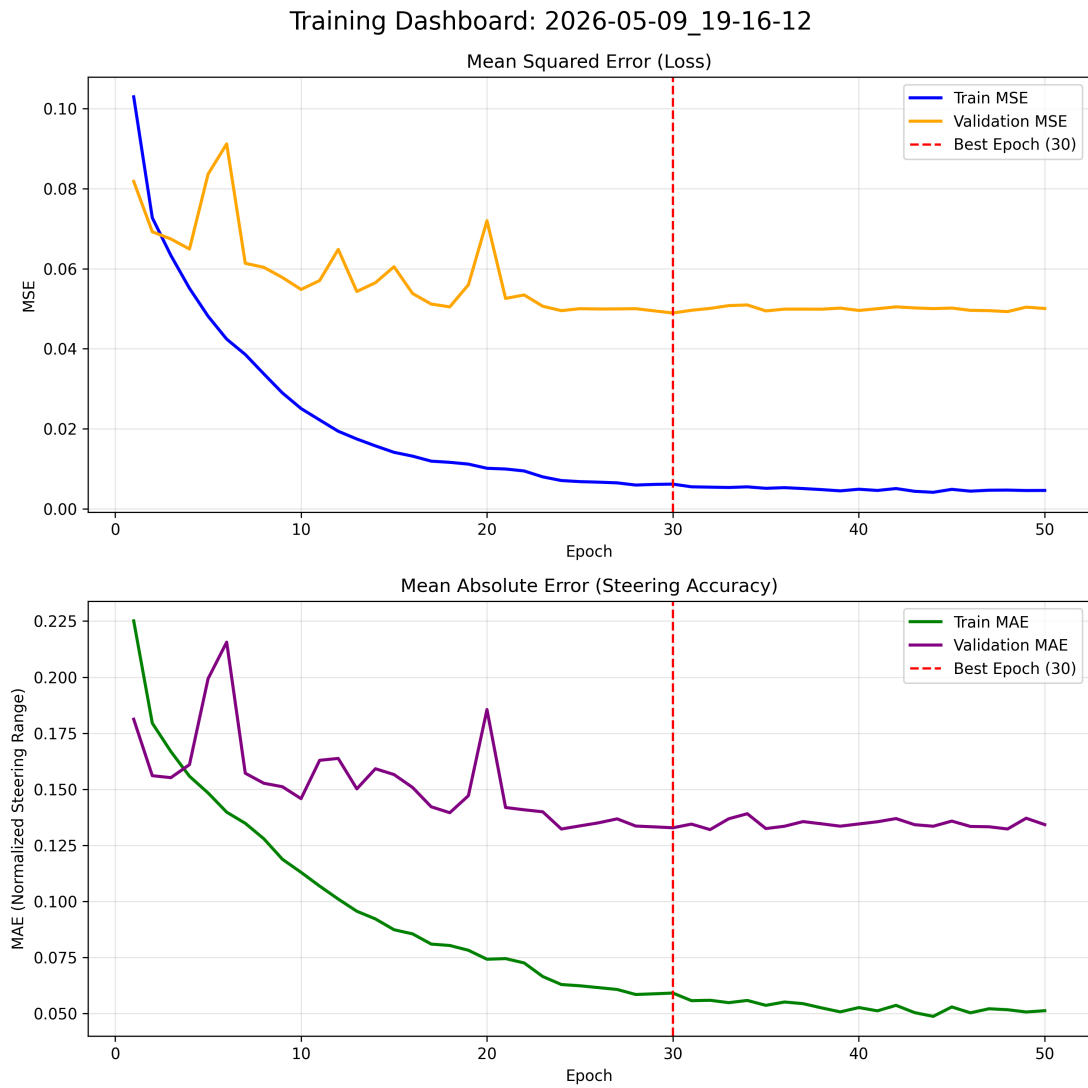


Figure 4.1: The near-perfect convergence of MSE and MAE suggested successful learning, though this was later identified as spatial memorization.

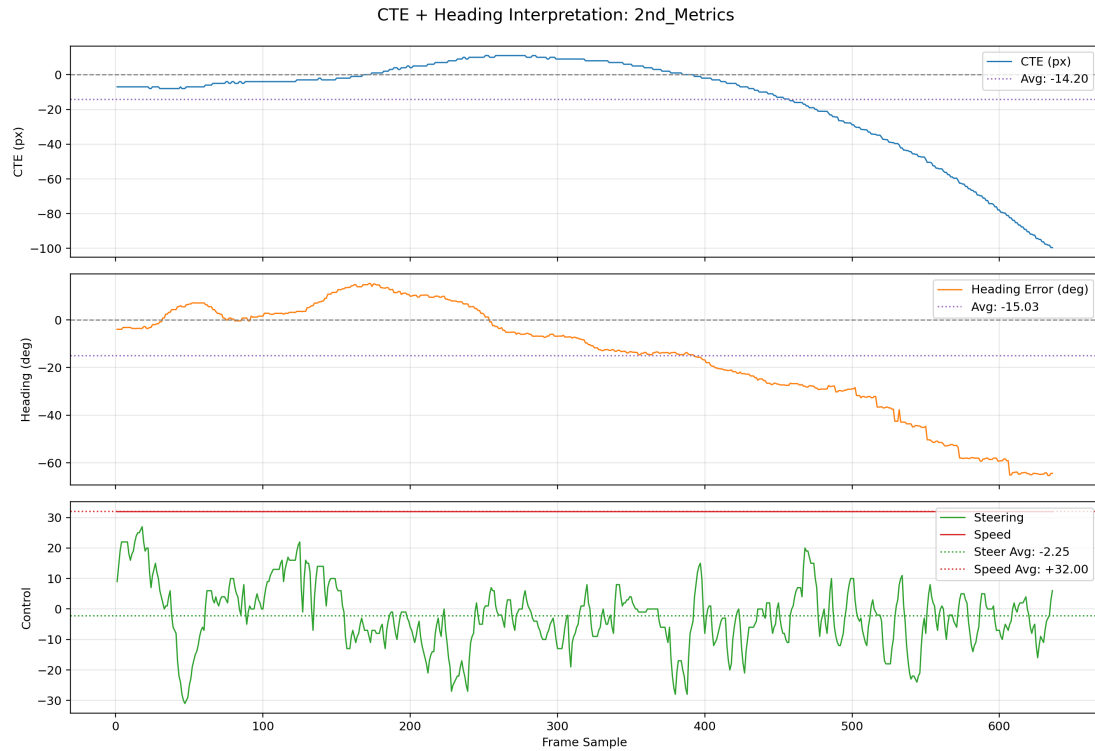


Figure 4.2: Physical deployment of the Allocentric Full AOI Model resulted in immediate navigation failure.

destabilized. As shown in Figure 4.3, the training profile exhibited severe validation noise and a massive loss spike near epoch 13, indicating the network struggled to map steering angles without its familiar background shortcuts. The resulting controller was not stable enough to justify full physical deployment, and the model was therefore not advanced beyond limited sanity checks.

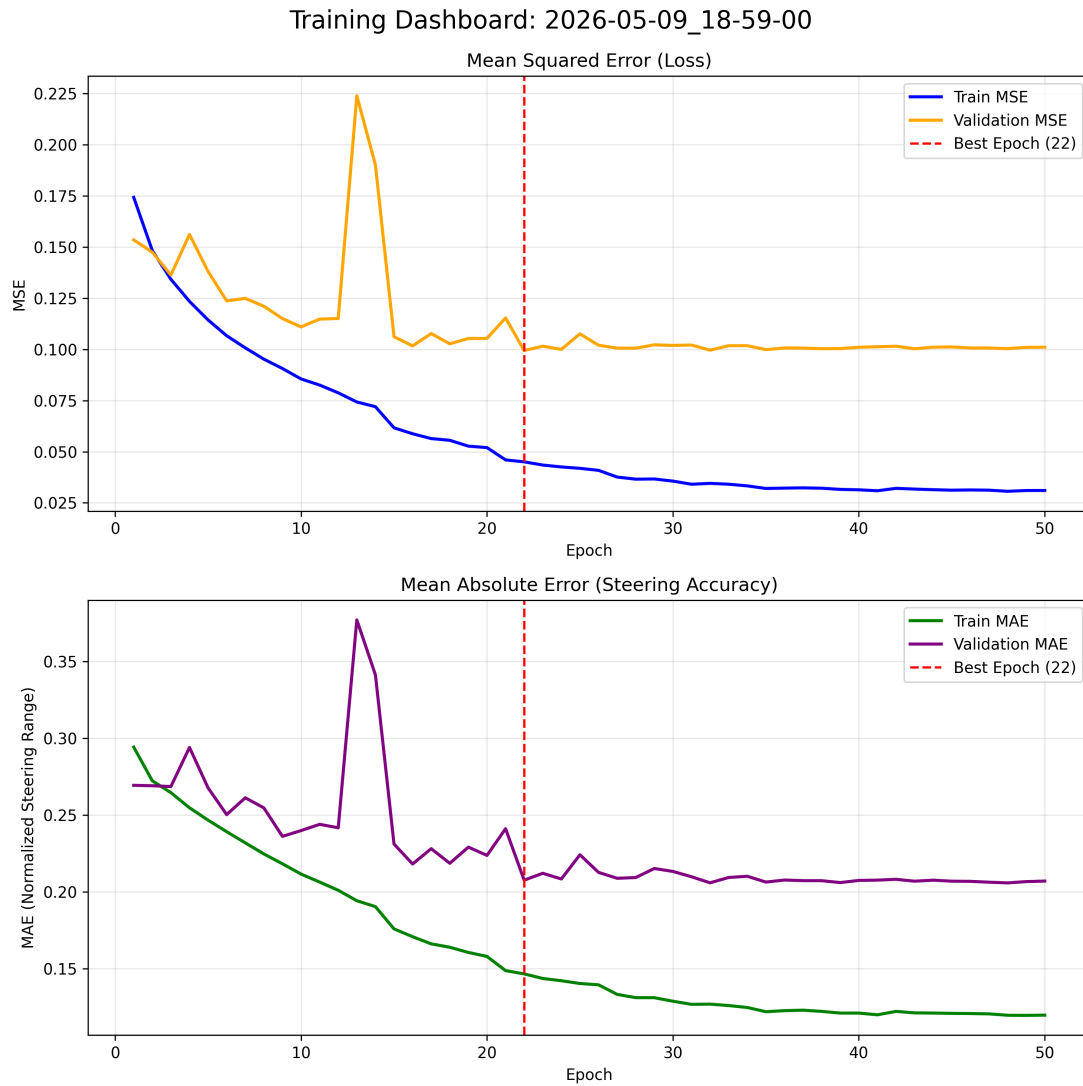


Figure 4.3: The Feature-Extracted Allocentric Model’s training profile was unstable, with severe validation noise and a massive loss spike, indicating the network struggled without background shortcuts.

Limited real-world trials at reduced speed still exposed the same instability. The failure was reproduced across two separate deployment sessions, in which the controller survived only 772 and 1,122 frames with CTE RMSE values of 35.72 px and 84.38 px respectively (Table 4.1), confirming that the instability was systematic rather than an isolated event. As shown in Figure 4.4 for the longer session, the Cross-Track

Error (CTE) drifted steadily upward to roughly 80 pixels before collapsing, while the Heading Error hovered around -20° and then jumped abruptly to approximately $+60^\circ$ near the failure point. The steering signal oscillated with a near-zero mean while speed was increased, indicating the controller was reacting to noise rather than tracking a stable path.

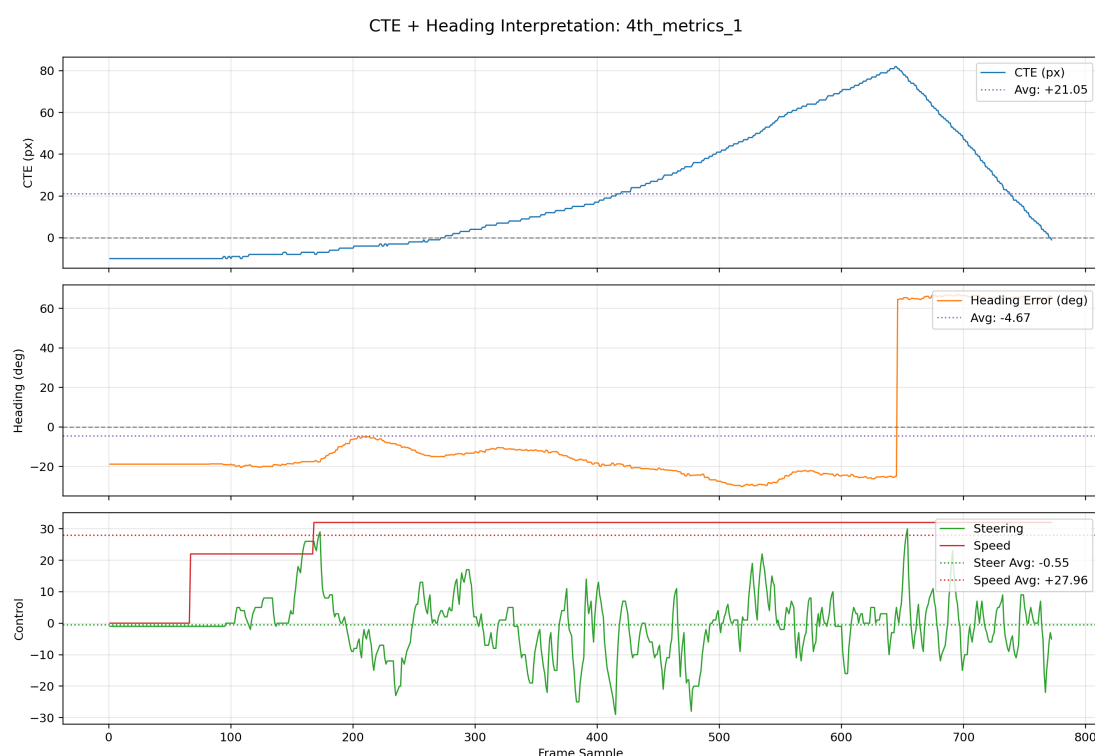


Figure 4.4: Limited real-world trial for Baseline B. CTE drifted upward with a late-stage heading discontinuity, while steering oscillations persisted under increasing speed.

Failure Analysis of Baselines A and B

The two allocentric baselines failed for complementary reasons that both trace back to the state representation. Baseline A preserved the full room context, which enabled shortcut learning: the CNN exploited static background cues and absolute pixel coordinates rather than learning the path geometry. This produced excellent offline metrics but catastrophic real-world generalization under even minor pose shifts.

Baseline B removed the environmental shortcuts, but the allocentric frame still did

not encode the robot's egocentric heading and local path curvature in a stable way. The mapping from a global, top-down AOI to a precise steering command became ambiguous and noise-sensitive, leading to unstable training dynamics and a controller that could not be stabilized for deployment.

Together, these failures established the need for a representation that is both egocentric and dynamically aligned to the robot's local path context, motivating the subsequent Egocentric Dynamic Crop Model.

4.2 | The Egocentric Dynamic Crop Model

To resolve both the shortcut learning of Iteration 1 and the spatial translation issues of Iteration 2, the final proposed model implemented the Egocentric Dynamic Crop.

By cropping the image continuously around the ArUco marker, the robot's position remains fixed at the center of the neural network's view. With the background removed, the path geometry simply rotates and approaches the robot, forcing the CNN to learn actual driving rules based strictly on path curvature.

4.2.1 | Offline Training Convergence

The proposed model resolved the previous instability issues. By centering the image on the robot, the network required only 9,679 samples to establish geometric stability.

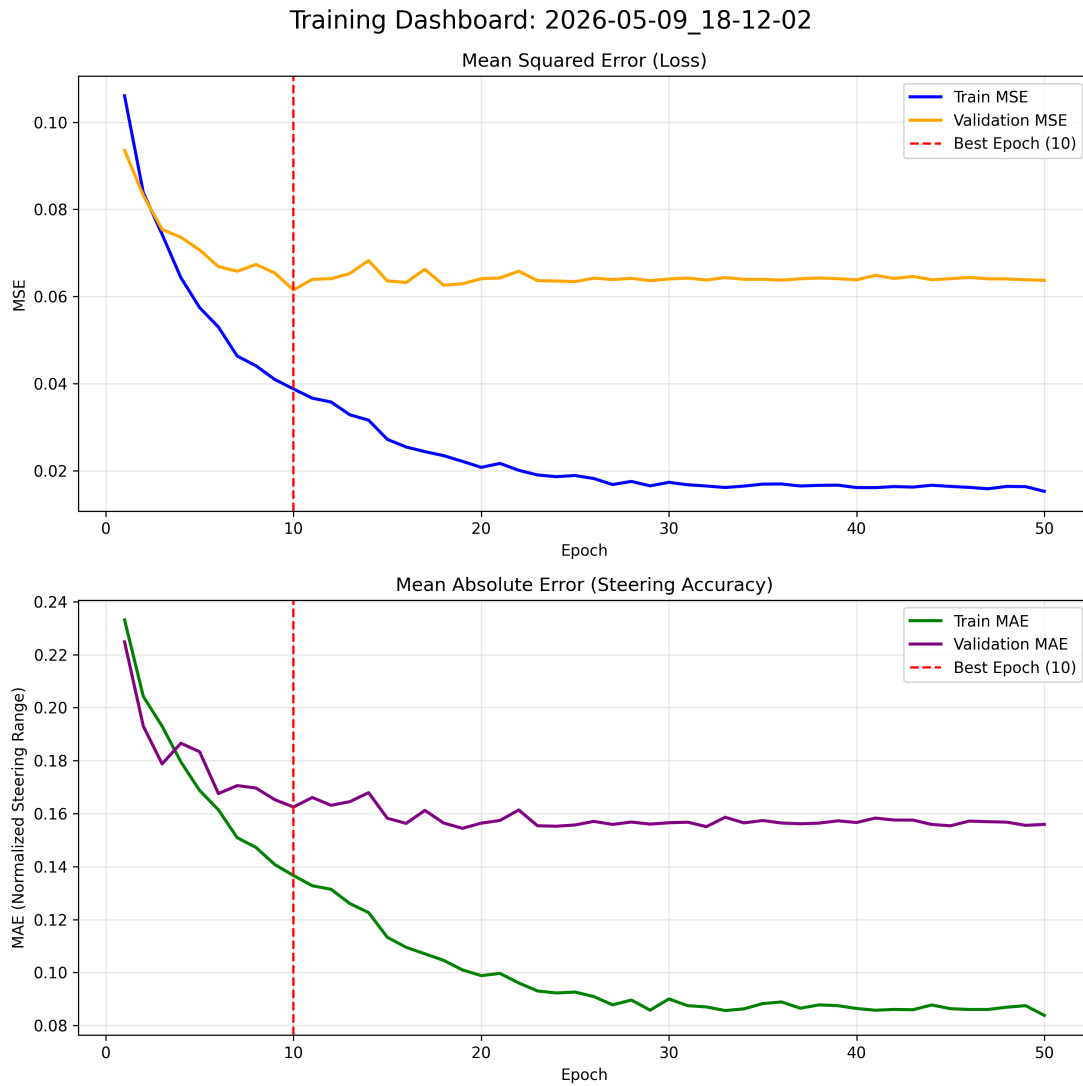


Figure 4.5: The Egocentric Dynamic Crop Model’s training profile showed stable convergence, with both MSE and MAE decreasing steadily without significant validation noise.

As shown in Figure 4.5, it quickly achieved a highly stable validation MSE of 0.0615 by Epoch 10. The entire optimization process was completed in just 2 minutes and 24 seconds, proving the model is both accurate and computationally lightweight.

4.2.2 | Real-World Navigation and Endurance

The proposed Egocentric Model successfully guided the robot through the entire track.

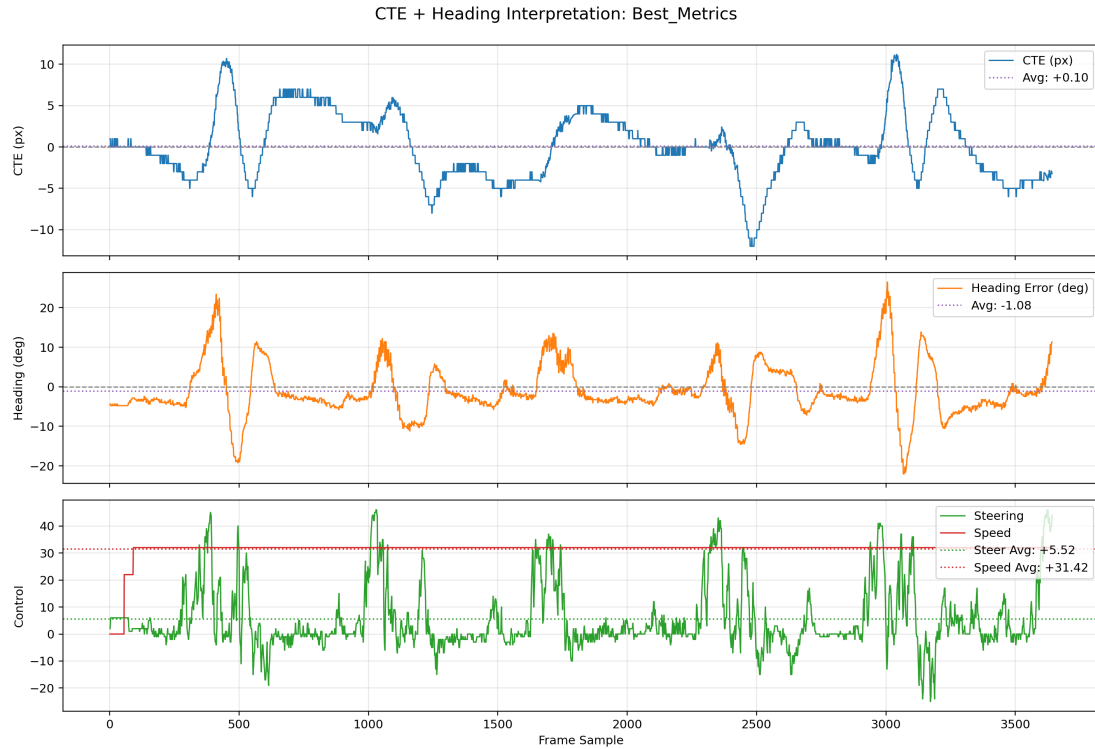


Figure 4.6: Physical deployment of the Egocentric Dynamic Crop Model resulted in successful navigation through the entire track, with stable CTE and Heading Error profiles.

Deployed over a continuous run of 3,642 frames, it achieved an extremely low tracking error with an RMSE of 4.0249 pixels and a Mean Absolute CTE of just 3.1700 pixels. The telemetry in Figure 4.6 demonstrates smooth, consistent steering across both straightaways and sharp turns without a single boundary breach.

4.3 | Comparative Tracking Performance

Table 4.1 and Figure 4.7 summarize the final real-world tracking metrics across all three chronological iterations, clearly illustrating the dramatic stability improvements

achieved by the egocentric crop.

Table 4.1: Physical Track Testing Results Across All Iterations

Physical Trial	Active Model	Total Frames Survived	Track Outcome	CTE (RMSE, px)
Baseline A	Baseline A (Full AOI)	636	Drove off track	30.9992
Baseline B (Session 1)	Baseline B (Feature-Extracted)	772	Drove off track	35.7224
Baseline B (Session 2)	Baseline B (Feature-Extracted)	1,122	Drove off track	84.3785
Final System	Proposed Egocentric Model	3,642	Completed successfully	4.0249

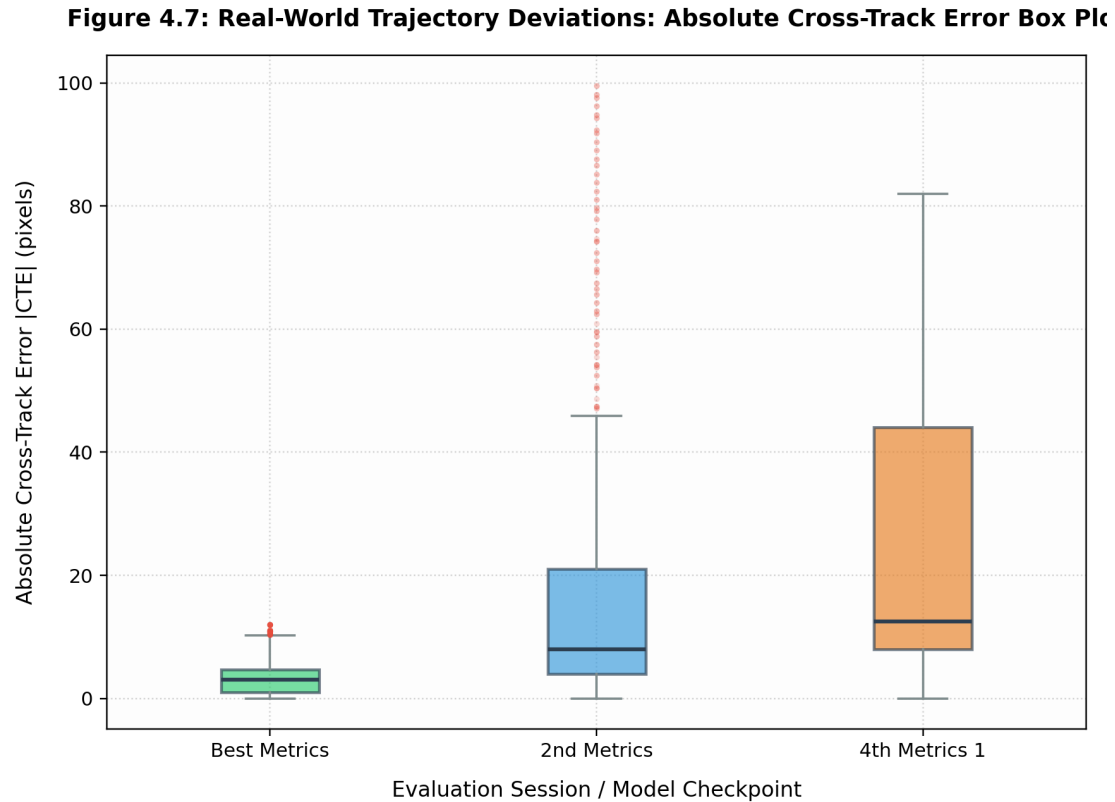


Figure 4.7: Real-World Trajectory Deviations: Absolute Cross-Track Error Box Plot.

The box plot in Figure 4.7 shows a dramatic contraction of the error distribution for the proposed model, with a much lower median and tighter interquartile range than Baselines A and B. This indicates not only lower average tracking error but also substantially reduced variability and fewer large deviations.

4.4 | In-Depth Performance of the Model

To comprehensively validate the proposed end-to-end architecture, the Egocentric Dynamic Crop model was evaluated across three specific track topologies of increasing Bézier curve complexity. By entirely removing the PID control components and relying strictly on the Convolutional Neural Network (CNN) for high-level navigation, this phase of testing isolates the controller's ability to dynamically scale continuous steering

outputs based solely on visual geometric features.

4.4.1 | Statistical Analysis Methodology

For each evaluation scenario, the robot was deployed in a continuous autonomous run during which the tracking system recorded per-frame Cross-Track Error (CTE) and Heading Error at approximately 60 Hz. To evaluate tracking consistency over sustained periods, telemetry is resolved per *lap* rather than over arbitrary fixed-length frame windows. A lap represents one complete physical circuit of the track, aligning each consistency window with a meaningful unit of the course rather than a time-slice that spans variable parts of a circuit. Laps are partitioned by dividing the total duration of each scenario by the number of complete laps. Mirroring the policy of excluding incomplete segments, partial trailing laps (e.g., the final 14.3 s of the Scenario 3 run) are discarded so that only complete circuits contribute. The full quantitative summary is presented in Tables 4.2, 4.3, and 4.4.

All cross-track errors are measured in the rectified bird’s-eye-view canvas, which spans 800×800 pixels and maps to the physical arena of approximately 2.0–3.0 m per side. Adopting a nominal arena width of 2.5 m yields a scale of approximately 0.31 cm per pixel, which is used throughout this chapter to report approximate real-world deviations alongside the pixel measurements. Because the physical arena dimension is approximate, the centimetre figures are likewise approximate (to within roughly $\pm 20\%$). The steering command reported throughout is the raw controller command after scaling the network’s normalized output (scaling gain $k_{steer} = 50$, clipped to $[-50, 50]$), where positive values denote rightward turns and negative values leftward turns. Commands between -2 and 2 are treated as straight-line driving (deadband). Empirically, this raw command maps monotonically to the robot’s measured yaw rate and saturates at high-magnitude commands as the differential-drive platform reaches its motor limits; the values are therefore reported as unitless command magnitudes rather than physical angles.

Table 4.2: Kinematic Performance Summary of the Final Egocentric Model

Scenario	Track Topology	Frames	Cross-Track Error (px)		
			Mean $\pm$ SD	RMSE	P95
1	45° Bézier curves	2,934	1.91 ± 1.84	2.66	5.0
2	90° square path	1,709	2.74 ± 2.23	3.54	8.0
3	180° sustained U-turn	13,421	5.07 ± 4.62	6.86	15.0

4.4.2 | Per-Lap Tracking Consistency

Because each evaluation run is a continuous, multi-lap circuit, tracking consistency is reported per *lap* rather than over arbitrary fixed-length frame windows. A lap is one complete circuit of the track, so a per-lap window aligns each consistency sample with a physically meaningful unit of the course instead of an 8.3 s slice that spans a variable fraction of a lap. The 45° and 90° runs each comprise 10 laps (lap periods of ≈ 4.9 s and ≈ 2.9 s respectively); the 180° endurance run comprises 8 complete laps, its two 180° turns per lap yielding a lap period of ≈ 26 s. Mirroring the earlier policy of discarding partial trailing windows, the 180° run’s final partial lap (14.3 s) is excluded so that only complete laps contribute. Per-lap RMSE summaries are given in Table 4.3, with the lap-by-lap values in Table 4.4.

Table 4.3: Per-Lap CTE RMSE Consistency (Complete Laps)

Scenario	Laps (<i>n</i>)	Lap dur. (s)	Per-Lap CTE RMSE (px)			
			Mean	SD	Min	Max
1 (45°)	10	≈ 4.9	2.28	1.37	0.93	4.73
2 (90°)	10	≈ 2.9	3.01	1.86	1.21	7.02
3 (180°)	8	≈ 26.0	6.56	2.08	4.43	10.14

Table 4.4: Lap-by-Lap CTE RMSE (px)

Scenario	L1	L2	L3	L4	L5	L6	L7	L8	L9	L10
1 (45°)	4.56	3.59	1.23	1.61	1.34	0.93	1.79	1.65	1.35	4.73
2 (90°)	1.58	1.23	1.62	1.21	4.49	7.02	4.55	3.96	2.99	1.43
3 (180°)	4.65	4.43	5.03	6.29	10.14	9.75	6.65	5.50	–	–

The lap-resolved view exposes structure that the run-level averages obscure. The 45° run shows an initial settling transient: the vehicle begins displaced from the path (laps 1–2, RMSE 4.56 and 3.59 px) and converges within two laps to tight tracking (laps 3–9 all below 1.8 px), with a single elevated final lap. Both the 90° and 180° runs exhibit a mid-run excursion followed by recovery rather than monotonic drift: error peaks near the middle of each run (90° laps 5–6, up to 7.02 px; 180° laps 5–6, up to 10.14 px) and then progressively returns to its earlier level by the final laps. This bounded, self-correcting behaviour indicates the controller does not diverge under sustained operation. Mean per-lap RMSE still scales with curvature severity ($2.28 < 3.01 < 6.56$ px for the 45°, 90°, and 180° courses).

4.4.3 | Scenario 1: 45-Degree Bézier Curves

The first scenario tests mild, continuous curvature over 2,934 frames (49.0 s) of continuous autonomous driving. Rather than exhibiting the fishtailing or oscillatory steering behavior commonly observed in pure behavioral cloning models without derivative damping, the model demonstrated highly anticipatory steering. As the 45° curves entered the egocentric bounding box, the convolutional feature extractor registered the gradual geometric shift, resulting in a smooth, continuous adjustment of the steering vector. The model achieved a mean absolute CTE of 1.91 ± 1.84 px (RMSE = 2.66 px, ≈ 0.83 cm), with 95% of all frames remaining within 5.0 px (≈ 1.6 cm) of the reference path. Per-lap consistency analysis across 10 completed laps yielded a mean per-lap RMSE of 2.28 ± 1.37 px (range: 0.93–4.73 px, Table 4.3), confirming consistent tracking throughout the run. The Heading Error remained bounded, with a mean of $-1.05 \pm 5.13^\circ$ and peaking at 31.92° at the sharpest point of the transition, proving that the model does not overreact to gentle path deviations and smoothly tracks shallow curves.

4.4.4 | Scenario 2: 90-Degree Bézier Corners (Square Path)

Navigating the square path over 1,709 frames (28.5 s) tested the model's response to rapid, orthogonal geometric changes. The model achieved a mean absolute CTE of 2.74 ± 2.23 px (RMSE = 3.54 px, ≈ 1.1 cm), with 95% of frames within 8.0 px (≈ 2.5 cm) of the reference path. The Heading Error remained bounded, with a mean of $-3.16 \pm 5.74^\circ$ and a peak magnitude of 20.98° . Approaching a 90° corner requires a sharp spike in actuation. Upon entering the apex of the corners, the model successfully executed aggressive, high-magnitude steering commands, reaching a peak raw steering command magnitude of **33.0**.

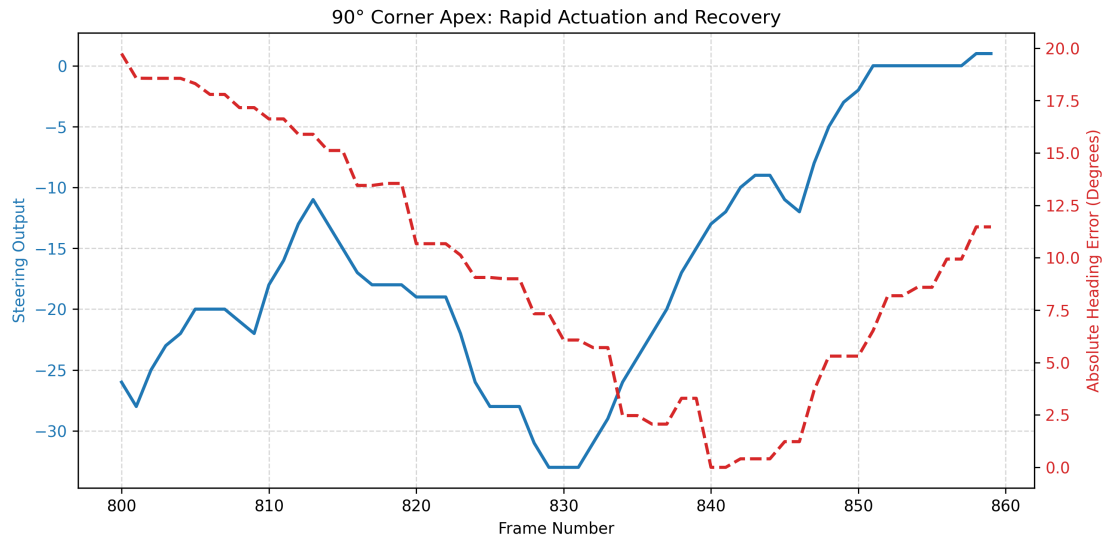


Figure 4.8: 90-degree cornering telemetry showing rapid actuation and recovery under orthogonal curvature.

Importantly, as visualized in Figure 4.8, the Egocentric Dynamic Crop ensured that the path boundaries remained centered in the network's view rather than sweeping out of frame. This continuous recentering allowed for a rapid and stable recovery; the model brought the heading error back down to 0° within just **11 frames** (approximately **0.73 s** at a 15 Hz operating frequency) upon exiting the corner, resuming straight-line stability instantly.

4.4.5 | Scenario 3: 180-Degree Sustained U-Turns

The final topology tested the system's kinematic endurance during extreme, continuous curvature. This scenario produced the longest continuous run at 13,421 frames over 226.5 s (approximately 3.8 minutes) of uninterrupted autonomous driving, encompassing many complete laps of the U-turn track. During the 180-degree U-turns, the model was forced to hold a near-maximum steering angle for a prolonged period.

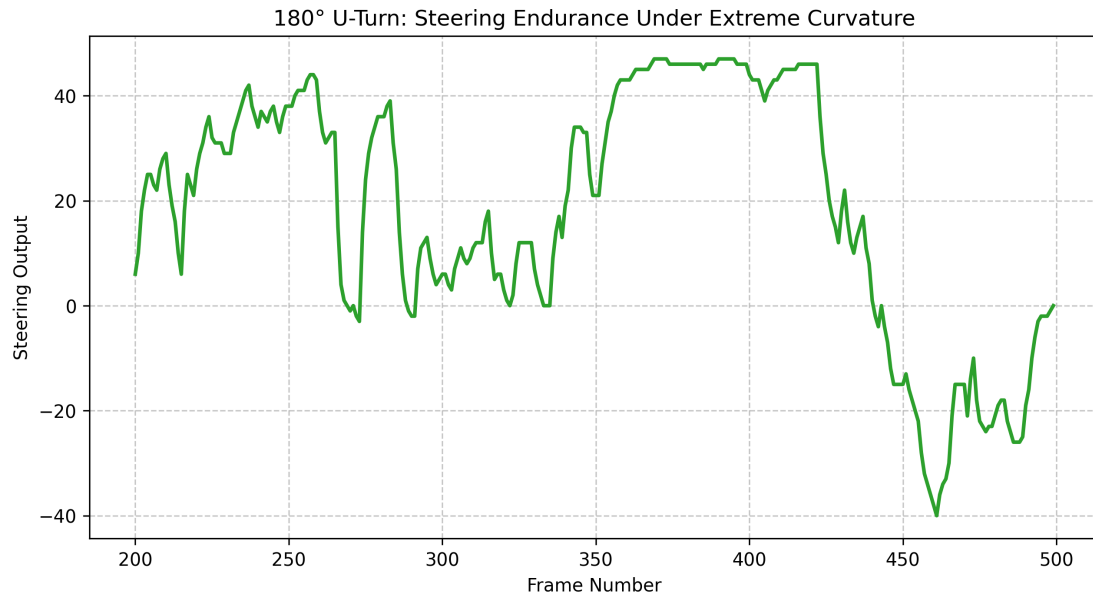


Figure 4.9: 180-degree U-turn telemetry illustrating sustained steering actuation under extreme curvature.

The telemetry in Figure 4.9 confirms that the network successfully handled the extreme geometric distortion, scaling up its actuation to reach and sustain a maximum raw steering command magnitude of **47.0**. The model achieved a mean absolute CTE of 5.07 ± 4.62 px (RMSE = 6.86 px, ≈ 2.1 cm), with 95% of frames within 15.0 px (≈ 4.7 cm) of the reference path and a worst-case deviation of 28.9 px (≈ 9.0 cm), which remains below half the chassis width. The Heading Error over the run had a mean of $-0.16 \pm 9.34^\circ$ with a peak magnitude of 33.95° . Per-lap consistency analysis across 8 completed laps yielded a mean per-lap RMSE of 6.56 ± 2.08 px (range: 4.43–10.14 px, Table 4.3), demonstrating that tracking performance remained bounded throughout the extended run despite the extreme curvature. This confirms that the CNN effectively learned the structural rules of extreme curvature tracking without requiring a rigid mathematical path-planning algorithm.

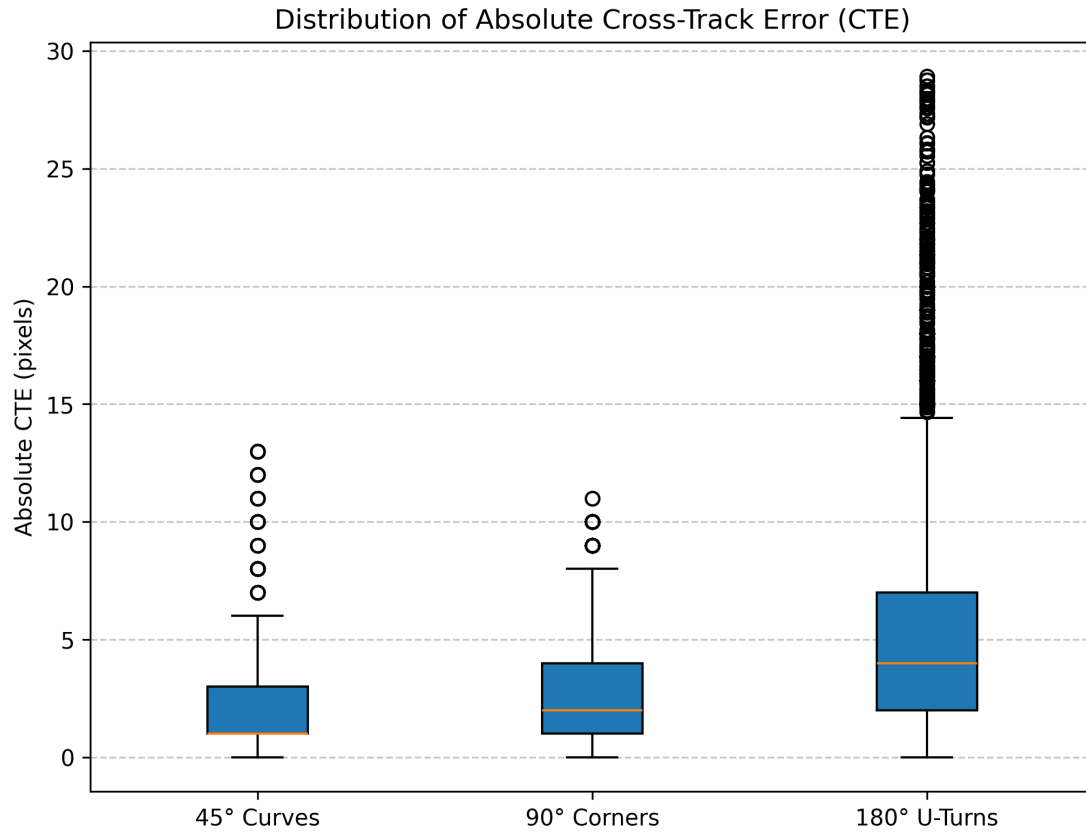


Figure 4.10: Distribution of Absolute Cross-Track Error (CTE) across varying Bézier curve complexities, demonstrating bounded error scaling under extreme geometric distortion.

4.5 | Control Decision Discretization and Confusion Matrix

To evaluate the directional correctness of steering decisions issued by the end-to-end CNN, the continuous telemetry logs were mapped to discrete control states. Under the system's sign convention, positive values denote rightward sense for both quantities: a positive CTE indicates the robot is displaced to the *right* of the reference path, and a positive steering command drives a rightward turn. Consequently, a positive CTE calls for a corrective *leftward* steering action. The desired corrective action is therefore defined from the lateral displacement (CTE) as:

$$\text{Desired Action} = \begin{cases} \text{Left,} & \text{if CTE} > 2.0 \text{ px} \\ \text{Straight,} & \text{if } -2.0 \text{ px} \leq \text{CTE} \leq 2.0 \text{ px} \\ \text{Right,} & \text{if CTE} < -2.0 \text{ px} \end{cases} \quad (4.1)$$

The executed CNN control action is discretized from the raw steering command using the same ± 2.0 dead-band, with commands greater than $+2.0$ classified as Right, those below -2.0 as Left, and those in between as Straight. With this convention, the matrix diagonal corresponds to correct corrective steering. The combined confusion matrix across all three track topologies is shown in Figure 4.11.

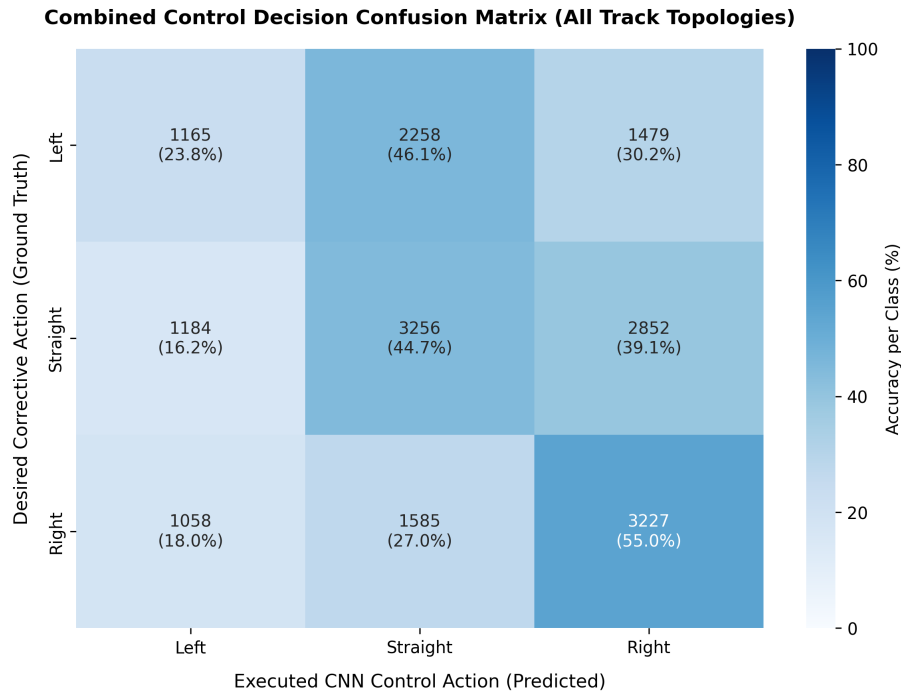


Figure 4.11: Combined Control Decision Confusion Matrix across all track topologies. The matrix maps cross-track error states to the corresponding steering actions.

A key control-theoretic observation is the *Proactive Curve-Following Paradox*. When tracking a curve accurately, the controller holds the vehicle near the centerline (ground truth classified as Straight) while still executing active left or right steering to follow the upcoming geometry. The resulting label mismatch does not indicate failure;

it demonstrates anticipatory, feedforward trajectory planning rather than a purely reactive corrective policy.

Calendar of Activities

Table A.1: Project Implementation Schedule and Timeline

Phase	Activity	Timeline
I. Hardware Setup		
	Hardware & system configuration	Jan Week 1–2
	Chassis assembly & wiring	Jan Week 2–3
	Camera mounting & calibration	Jan Week 2–3
II. Software Development		
	OS & environment setup	Jan Week 3–4
	Dev: perception (homography)	Jan Week 4–Feb Week 6
	Dev: data logger module	Feb Week 6–7
III. Data Pipeline		
	Dataset collection (driving)	Feb Week 7–Mar Week 9
	Data cleaning & augmentation	Mar Week 10–11
IV. Model Development		
	CNN architecture design & dev.	Mar Week 12–13
	Model training (GPU)	Apr Week 13–14
	Hyperparameter tuning	Apr Week 14–15
V. Integration & Testing		
	Deployment to robot	May Week 14–15
	Scenario testing (A, B, C)	May Week 15–17
	Analysis & documentation	May Week 17

Personal Vitae

John Christian Niño T. Abuel

- Title: 4th Year Bachelor of Science in Computer Science
- Affiliation: Mindanao State University – Iligan Institute of Technology
- Address: Block 2 Lot 27 Doña Maria Subdivision Phase 5, Tipanoy, Iligan City
- Contact Number: 0945 702 5852
- Email Address: johnchristiannino.abuel@g.msuiit.edu.ph

Kervin Lemuel Paalisbo

- Title: 4th Year Bachelor of Science in Computer Science
- Affiliation: Mindanao State University – Iligan Institute of Technology
- Address: Poblacion Linamon Purok 5-A, Lanao Del Norte
- Contact Number: 09924779137
- Email Address: kervinlemuel26@gmail.com

Rey Iann Tigley

- Title: 4th Year Bachelor of Science in Computer Science

- Affiliation: Mindanao State University – Iligan Institute of Technology
- Address: Prk Sudlonon, San Miguel, Iligan City, 9200 Lanao del Norte
- Contact Number: 09605052173
- Email Address: reyiann.tigley@g.msuiit.edu.ph